Check for updates

Weifei Hu
Mem. ASME
State Key Laboratory of Fluid Power and Mechatronic Systems,
Zhejiang University,
Hangzhou 310027, China
e-mail: weifeihu@zju.edu.cn

Yihan He
School of Mechanical Engineering,
Zhejiang University,
Hangzhou 310027, China
e-mail: yihanh@126.com

Zhenyu Liu[1]
State Key Laboratory of Fluid Power and Mechatronic Systems,
Zhejiang University,
Hangzhou 310027, China
e-mail: liuzy@zju.edu.cn

Jianrong Tan
State Key Laboratory of Fluid Power and Mechatronic Systems,
Zhejiang University,
Hangzhou 310027, China
e-mail: zjume@zju.edu.cn

Ming Yang
Xiamen Intretech Inc.,
No. 100, Dongfu West Road, Haicang District,
Xiamen 0582841, Fujian, China
e-mail: ming_yang@intretech.com

Jiancheng Chen
Xiamen Intretech Inc.,
No. 100, Dongfu West Road, Haicang District,
Xiamen 0582841, Fujian, China
e-mail: xmcjc@intretech.com

Toward a Digital Twin: Time Series Prediction Based on a Hybrid Ensemble Empirical Mode Decomposition and BO-LSTM Neural Networks

Precise time series prediction serves as an important role in constructing a digital twin (DT). The various internal and external interferences result in highly nonlinear and stochastic time series. Although artificial neural networks (ANNs) are often used to forecast time series because of their strong self-learning and nonlinear fitting capabilities, it is a challenging and time-consuming task to obtain the optimal ANN architecture. This paper proposes a hybrid time series prediction model based on an ensemble empirical mode decomposition (EEMD), long short-term memory (LSTM) neural networks, and Bayesian optimization (BO). To improve the predictability of stochastic and nonstationary time series, the EEMD method is implemented to decompose the original time series into several components (each component is a single-frequency and stationary signal) and a residual signal. The decomposed signals are used to train the neural networks, in which the hyperparameters are fine-tuned by the BO algorithm. The following time series data are predicted by summating all the predictions of the decomposed signals based on the trained neural networks. To evaluate the performance of the proposed EEMD-BO-LSTM neural networks, this paper conducts two case studies (the wind speed prediction and the wave height prediction) and implements a comprehensive comparison between the proposed method and other approaches including the persistence model, autoregressive integrated moving average (ARIMA) model, LSTM neural networks, BO-LSTM neural networks, and EEMD-LSTM neural networks. The results show an improved prediction accuracy using the proposed method by multiple accuracy metrics.
[DOI: 10.1115/1.4048414]

Keywords: digital twin, time series prediction, LSTM neural networks, mode decomposition, Bayesian optimization, wind speed, wave height, artificial intelligence, design optimization, design theory and methodology, machine learning, multidisciplinary design and optimization, wind energy

1 Introduction

While a digital twin (DT) has been gradually evolved from a conceptual mirrored spaces model used in product lifecycle management (PLM) [1] to a model-based definition for DT [2], as well as many other definitions and frameworks (e.g., see in Refs. [3–7]), it generally refers to a digital replica of physical assets, processes, people, places, and systems that provides both the elements and the dynamics of how an Internet of things (IoT) device operates and lives throughout its lifecycle. Instead of proposing a new definition or framework of DTs, this paper is dedicated to present a time series prediction approach based on a hybrid ensemble empirical mode decomposition (EEMD) and Bayesian optimization—long short-term memory (BO-LSTM) neural networks, which could be one of the most critical steps toward a DT.

The DT has drawn significant attention recently for its promising values that could never have been realized prior to the advent of connected smart technologies [4,5,8,9]. For instance, the DT has been adopted as a conceptual basis in the astronautics and aerospace field. The concept has been proposed for the future generation of NASA and U.S. Air Force vehicles integrating ultra-high fidelity simulation, a vehicle health management system, maintenance history, and all available historical and fleet data to mirror the life of the flying twin and enable unprecedented levels of safety and reliability [3,6]. In mechatronic and cyberphysical systems (CPS), a DT allows for information created during the design to also be available and ready for evaluation during the operation of the system; from the point of view of data usage, such information is currently often neglected, as the design and operation are mainly disconnected from the lifecycle phases [7]. Frameworks of DT shop floors have been proposed for the smart production management and control of designing and manufacturing processes, e.g., satellite assembly [10], hollow-glass production [11], and human–robot collaboration [12]. Although various DT frameworks have been proposed for the all product lifecycle phases including design, manufacturing, and operation and maintenance (O&M), the intensive investigation of specific technologies, for example, time series prediction, for constructing a DT is often neglected.

Indeed, an efficient and accurate time series prediction is one of the key technologies for constructing a DT. By applying the time-series prediction function of a DT, any accessories' behavior in a complex equipment will be promptly predicted before the change actually happens, such that catastrophic or unfavorable consequences may be avoided. For example, by accurately predicting wind gust using a DT, wind turbines can be shut down before the extremely large wind gust is encountered to minimize the structural

[1]Corresponding author.

Contributed by the Design Automation Committee of ASME for publication in the Journal of Mechanical Design. Manuscript received February 23, 2020; final manuscript received July 28, 2020; published online November 17, 2020. Assoc. Editor: Christopher Hoyle.

 Copyright © 2020 by ASME

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vJFOndGsi5AAAAAA:CzOjbZGfofMEOE4IdIDVjkE6meTooSJL7kQeQE7RMgFjvSYmOmuKG5gly5ahvRgSKx7g by Universita Di Pisa user on 28 November

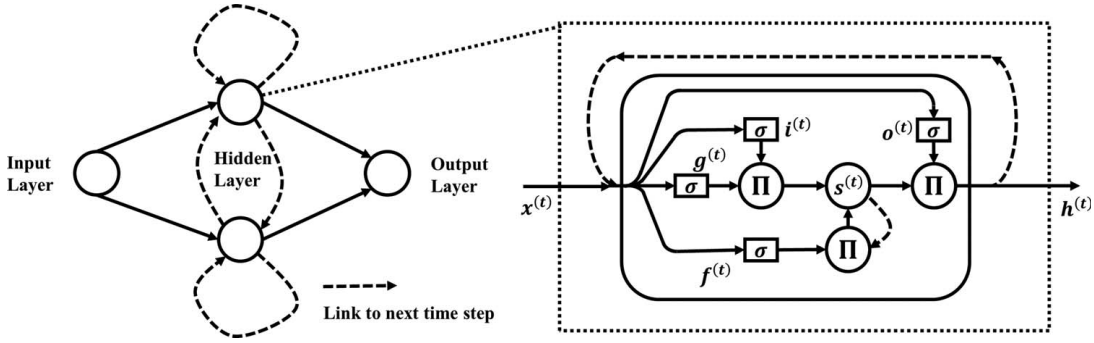


Fig. 1 A simple recurrent neural network (RNN) (left) and a LSTM cell (right), as proposed in Ref. [25]. In RNNs, a dashed link connects a source node at each time t to a target node at the following time $t+1$. In LSTM neural networks, each node in the hidden layer is replaced by an LSTM cell as illustrated.

and fatigue damage. A DT with an accurate and efficient time series prediction technology allows designers and O&M managers to create vivid simulation scenarios to effectively apply simulation tests on prototypes and accurately predict the actual performance of the physical products as far as possible [13]. The time series prediction of the performance and efficiency of a complex equipment, as a key part of a DT, could be used throughout all phases of the complex equipment's lifecycle from the design phase to the manufacturing phase, and to the O&M phases. However, this lifecycle performance, e.g., the structural response in the design phase, the assembly efficiency in the manufacturing phase, and the remaining fatigue life in the operation phase, is currently often poorly forecasted, as different phases are disconnected in traditional modeling and simulation processes. Moreover, time series prediction is of critical importance to develop the intelligent real-time and leading-time perception, which is viewed as one of the challenging technologies needed to be researched for implementing DT-driven product design, manufacturing, and service [13].

Although there are numerous time series prediction methods in the literature, the state-of-the-art methods may be categorized into four groups, neural prediction methods, statistical prediction methods, evolutionary prediction methods, and hybrid prediction methods. Neural prediction methods apply various artificial neural network (ANN) algorithms, inspired by the biological neural networks that constitute human brains, for solving a wide range of engineering problems [14]. For example, an artificial wavelet neural network, which combines wavelet decomposition and ANNs, has been proposed for predicting short-term wind speed [15].

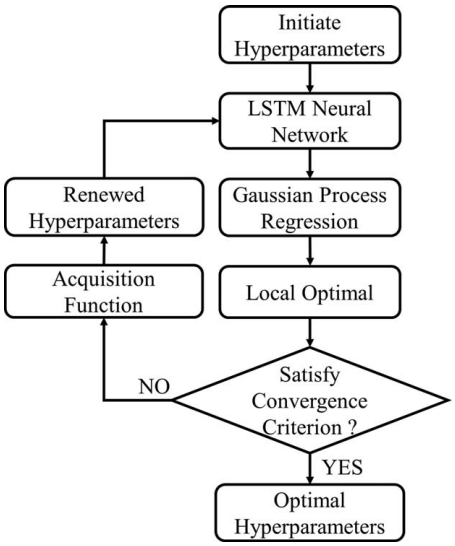


Fig. 2 Bayesian optimization for the hyperparameters of LSTM neural networks

Statistical prediction methods, also named as direct random time-series prediction methods, create mathematical models using pattern identification, parameter estimation, and model checking based on historical data. The mathematical models are often classified as a moving average (MA) model, autoregressive (AR) model, autoregressive moving average (ARMA) model, and autoregressive integrated moving average (ARIMA) model [16]. The statistical models have been created to represent time series of dynamic systems (e.g., a third-order AR model for stochastic road elevation process [17] and an ARIMA model for wind speed forecasting used in blade structural and fatigue analysis [18]). However, as univariate models, traditional ARIMA-type models have also been criticized, when used for predicting product returns, since the models could not utilize the information of past sales [19].

Evolutionary prediction methods are generic population-based metaheuristic methods using mechanisms inspired by biological evolution, such as reproduction, mutation, recombination, and selection. This kind of method engages a set of general stochastic searching methods and is a subset of evolutionary computation methods. Typical evolutionary prediction methods include a genetic algorithm, genetic programming, evolutionary programming, and evolution strategy. Lukoseviciute and Ragulskis use evolutionary methods to identify a near-optimal set of time lags for forecasting time series [20]. A covariance matrix adaptation evolutionary strategy is proposed to optimize the overall architecture and the parameters of an ARMA model and a neural network to predict time series of natural data and computer-generated sequences [21].

The above three types of prediction methods do not strictly repel each other. Actually, the marriage of different types of prediction methods, i.e., hybrid prediction methods, could potentially generate more accurate prediction results than the use of a single method. For example, hybrid models consisting of ARIMA models and ANN models have been developed in which the ANN models are built taking into account the nonlinear tendencies of errors that the ARIMA models cannot identify [22].

This paper proposes a new hybrid prediction approach integrating LSTM neural networks, BO, and EEMD. Unlike standard feed-forward neural networks, the LSTM neural network has feedback connections and is well-suited to time series prediction as it allows lags of unknown duration between important events in a time series. The performance of a neural network is largely determined by its architecture, which is controlled by hyperparameters. The process to achieve the best performance of the model by fine-tuning hyperparameters can be seen as an optimization problem, which is effectively solved by BO [23]. To further improve the prediction accuracy, an EEMD method is applied to decompose a stochastic and nonstationary time series into a series of intrinsic mode functions (IMFs) that are smooth, steady, and without mode-mixing, thus, enhancing the prediction accuracy [24].

The remaining structure of the paper is organized as follows. Section 2 explains the theoretical aspects of the time series prediction using LSTM neural networks, BO-LSTM neural networks, and

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vIFOndGsI5AAAAAA:CzOjbZGfofMEOE4IdIDVjkE6meT0oSjL7kQeQE7RMgFjvSYmOmuKG5gIyi5ahvRgSKx7g by Universita Di Pisa user on 28 November

EEMD-BO-LSTM neural networks. Case studies of wind speed and wave height prediction for constructing DTs of wind energy systems and offshore structures, respectively, are provided in Sec. 3, which is followed by the prediction results and discussion in Sec. 4. Concluding remarks and future work are presented in Sec. 5.

2 Theory

Time series prediction consists of creating a model to predict the future value(s) using the past values. Considering a time series $[x_1, \ldots, x_N]$ composed of N observations, a single-step-ahead prediction computes the immediately following value by a linear or nonlinear function T

$$x_{N+1} = T(x_N, \ldots, x_{N-d+1}) \tag{1}$$

where d is the embedding dimension that is the number of past values taken into consideration to predict one future value. In contrast, multi-step-ahead prediction consists of predicting $[x_{N+1}, \ldots, x_{N+h}]$, where $h > 1$ is the prediction horizon. Traditionally there are two families of prediction strategies for multi-step-ahead prediction: the single-output strategy, which relies on the estimation of a single-output predictor, and the multiple-output strategy, which learns from data with a multiple-input and multiple-output dependency between a window of past observations and a window of future values [25]. The advantage of the single-step-ahead prediction is that it does not propagate errors from the previous predictions, although it often requires a large amount of data to train one accurate prediction model. In addition, the multi-step-ahead prediction obtains a longer leading time than that of the single-step-ahead prediction. However, the former may accumulate the prediction errors (e.g., the iterated method in Ref. [25]) or require complex learning process to directly calculate the multiple predictions (e.g., the direct method in Ref. [25]). One of the key characteristics of a DT is that there is real-time data communication between the real space and the virtual space. The measurements or observations are instantly collected by sensors and fed to the DT system for data analysis. In this paper, we focus on developing a robust one-step-ahead prediction model for constructing a DT that enables time series forecasting based on the proposed EEMD-BO-LSTM neural networks method.

2.1 Long Short-Term Memory Neural Networks.

Long short-term memory (LSTM) neural networks are developed based on recurrent neural networks (RNNs), which are feedforward neural networks augmented by the inclusion of adjacent time-steps to form recurrent edges (or links, Fig. 1). The recurrent edges may form cycles, including cycles of length one that are self-connections from a node to itself across time. At time t, both the current input data $\mathbf{x}^{(t)}$ and the previous hidden node values $\boldsymbol{h}^{(t-1)}$ are fed into the hidden node values $\boldsymbol{h}^{(t)}$ with recurrent edges. Thus, the output $\hat{y}^{(t)}$ at each time t can be calculated from the hidden node values $\boldsymbol{h}^{(t)}$ at time t and are also influenced by input $\mathbf{x}^{(t-1)}$ at time $t-1$. This process can be mathematically expressed for a simple RNN as follows [26]:

$$\boldsymbol{h}^{(t)} = S(\boldsymbol{W}^{hx}\boldsymbol{x}^{(t)} + \boldsymbol{W}^{hh}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_h) \tag{2}$$

$$\hat{\boldsymbol{y}}^{(t)} = \phi(\boldsymbol{W}^{yh}\boldsymbol{h}^{(t)} + \boldsymbol{b}_y) \tag{3}$$

where $S()$ is a sigmoid activation function for the hidden nodes; $\boldsymbol{W}^{hx}$ is a matrix of the conventional weights between the input and the

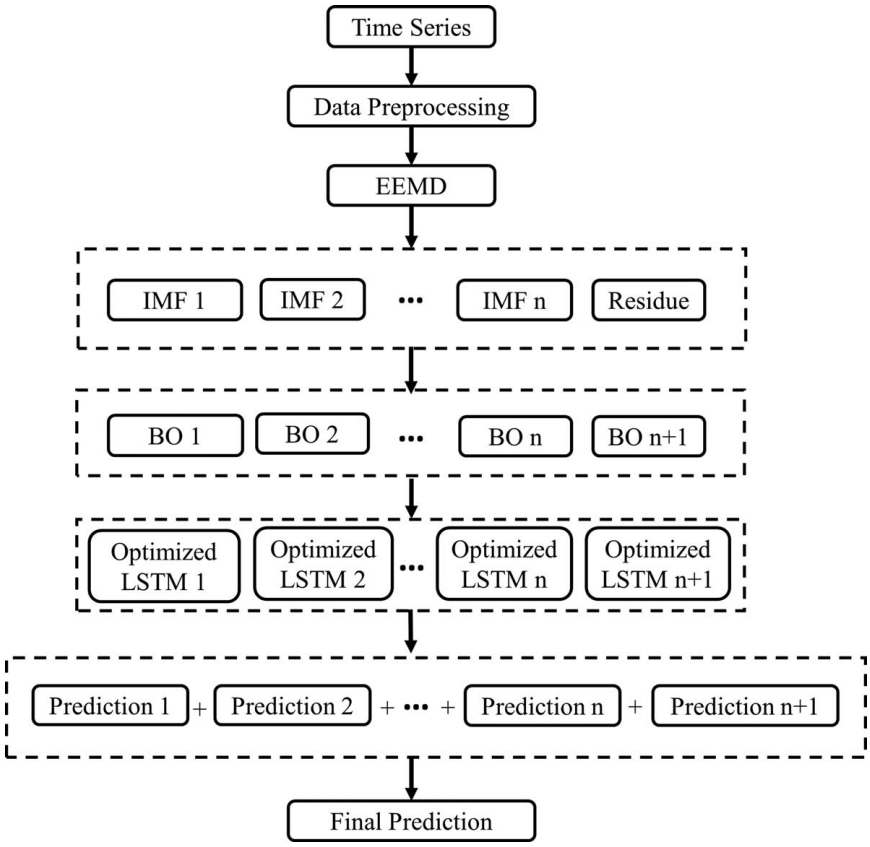


Fig. 3 A flowchart of the hybrid EEMD-BO-LSTM neural networks approach for time series prediction

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vIFOndGsi5AAAAAA:CzOjbZGfofMEOE4IdIDVjkE6meT0oSjL7kQeQE7RrMgFjvSYmOmuKG5gJy5ahvRgSKx7g by Universita Di Pisa user on 28 November

hidden layer; $\boldsymbol{W}^{hh}$ is a matrix of the recurrent weights between the hidden layer and itself at adjacent time-steps; $\boldsymbol{W}^{yh}$ is a matrix of the weights between the output and the hidden layer; $\boldsymbol{b}_h$ and $\boldsymbol{b}_y$ are the vectors of the bias parameters allowing each node to learn an offset for the hidden layer and the output layer, respectively; and $\varphi()$ is a tanh function for the output.

It can be especially challenging to train an RNN due to the difficulty of learning long-range dependencies, as explained in Ref. [27]. In addition, the problems of vanishing and exploding gradients occur when back-propagating errors across time-steps in early RNN designs have long been considered to be difficult to resolve [26,28]. To overcome these problems, Hochreiter and Schmidhuber first introduced the LSTM neural network, in which each ordinary node in the hidden layer of an RNN is replaced by a memory cell (Fig. 1) [29]. A self-connected recurrent edge of fixed weight one is added to the memory cell such that the gradient can pass across many time-steps without vanishing or exploding. Intuitively, the term "long short-term memory" is twofold: it inherits the long-term memory from the traditional RNN in the form of weights and also has a short-term memory in the form of ephemeral activation, which passes from each node to successive nodes [26].

A modern LSTM cell often has five elements, input node $\boldsymbol{g}^{(t)}$, input gate $\boldsymbol{i}^{(t)}$, internal state $\boldsymbol{s}^{(t)}$, forget gate $\boldsymbol{f}^{(t)}$, and output gate $\boldsymbol{o}^{(t)}$, as described in Refs. [26,30]. Along with the depiction in Fig. 1, the data flow in each element is briefly explained as follows. At the current time-step, the input node $\boldsymbol{g}^{(t)}$ receives activation in the standard way from the input layer $\mathbf{x}^{(t)}$ and from the hidden layer at the previous time-step $\boldsymbol{h}^{(t-1)}$. When multiplying the value of input node $\boldsymbol{g}^{(t)}$, the input gate $\boldsymbol{i}^{(t)}$, as a distinctive feature of the LSTM neural network, can cut off (when its value is 0) or pass through (when its value is 1) the data flow. The internal state $\boldsymbol{s}^{(t)}$ is at the heart of a memory cell and has a self-connected recurrent edge with fixed unit weight, which allows the error flow across time-steps without vanishing or exploding. The forget gate $\boldsymbol{f}^{(t)}$ then provides a method by which the network can learn to flush the content of the internal state from the previous time-step. Finally, the value of the output gate $\boldsymbol{o}^{(t)}$ multiplies the value of the internal state $\boldsymbol{s}^{(t)}$ to produce the ultimate value of the LSTM cell. A detailed explanation of the aforementioned five elements is well elaborated by Lipton et al. [26]. The computation in each element of the LSTM cell can be expressed in the following

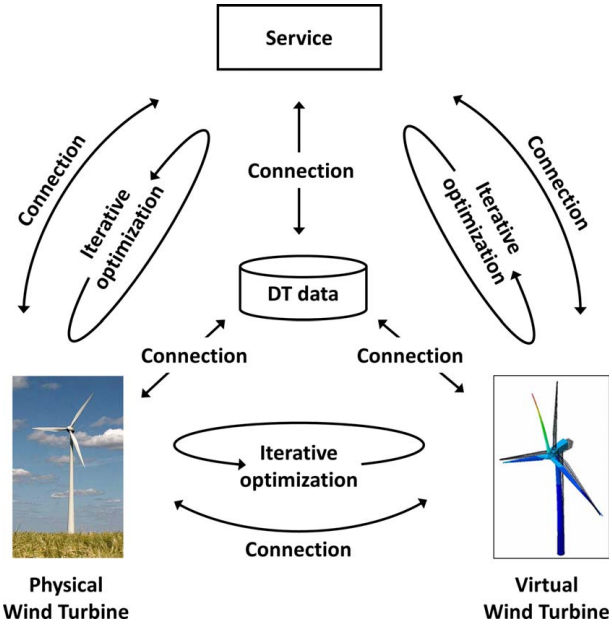


Fig. 4 Five-dimensional DT model for a wind turbine

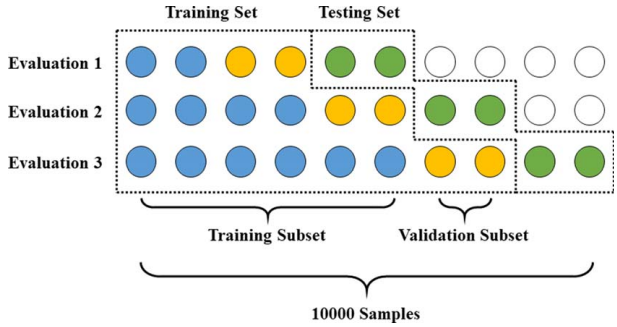


Fig. 5 Rolling-origin-recalibration threefold evaluation used in the case study of wind speed prediction

equations, which are calculated at every time-step

$$\boldsymbol{g}^{(t)} = \phi(\boldsymbol{W}^{gx}\boldsymbol{x}^{(t)} + \boldsymbol{W}^{gh}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_g) \tag{4}$$

$$\boldsymbol{i}^{(t)} = S(\boldsymbol{W}^{ix}\boldsymbol{x}^{(t)} + \boldsymbol{W}^{ih}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_i) \tag{5}$$

$$\boldsymbol{f}^{(t)} = S(\boldsymbol{W}^{fx}\boldsymbol{x}^{(t)} + \boldsymbol{W}^{fh}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_f) \tag{6}$$

$$\boldsymbol{o}^{(t)} = S(\boldsymbol{W}^{ox}\boldsymbol{x}^{(t)} + \boldsymbol{W}^{oh}\boldsymbol{h}^{(t-1)} + \boldsymbol{b}_o) \tag{7}$$

$$\boldsymbol{s}^{(t)} = \boldsymbol{g}^{(t)} \odot \boldsymbol{i}^{(t)} + \boldsymbol{s}^{(t-1)} \odot \boldsymbol{f}^{(t)} \tag{8}$$

$$\boldsymbol{h}^{(t)} = \phi(\boldsymbol{s}^{(t)}) \odot \boldsymbol{o}^{(t)} \tag{9}$$

where $\varphi()$ is a tanh activation function; $\boldsymbol{W}$'s are the weight matrices for different LSTM elements; $\boldsymbol{b}_g$, $\boldsymbol{b}_i$, $\boldsymbol{b}_f$, and $\boldsymbol{b}_o$ are the bias vectors for the input node, input gate, forget gate, and output gate, respectively; and $\odot$ indicates pointwise multiplication. The value eventually produced by a LSTM cell $\boldsymbol{h}^{(t)}$ is the value of the tanh activation function of the value of the internal state $\boldsymbol{s}^{(t)}$ multiplied by the value of the output gate $\boldsymbol{o}^{(t)}$, as shown in Eq. (9). After the values of all LSTM cells are calculated, the output of the LSTM neural network $\hat{\mathbf{y}}^{(t)}$ can then be calculated by Eq. (3).

For constructing a time series prediction model using the LSTM neural network, the input $\mathbf{x}^{(t)}$ and output $\mathbf{y}^{(t)}$ for training the model are actually the previous observations and the one-step-ahead observations, respectively

$$\boldsymbol{x}^{(t)} = [\boldsymbol{x}_N, \ldots, \boldsymbol{x}_{N-d+1}] \tag{10}$$

$$\boldsymbol{y}^{(t)} = \boldsymbol{x}_{N+1} \tag{11}$$

In this paper, the LSTM neural networks are trained with the Adam algorithm [31], in which the weight $\boldsymbol{W}$ is updated by the following equations:

$$\boldsymbol{G}_t = \nabla_{\boldsymbol{W}} J_t(\boldsymbol{W}_{t-1}) \tag{12}$$

$$\boldsymbol{v}_{1,t} = \beta_1 \boldsymbol{v}_{1,t-1} + (1-\beta_1)\boldsymbol{G}_t \tag{13}$$

$$\boldsymbol{v}_{2,t} = \beta_2 \boldsymbol{v}_{2,t-1} + (1-\beta_2)\boldsymbol{G}_t^2 \tag{14}$$

$$\hat{\boldsymbol{v}}_{1,t} = \frac{\boldsymbol{v}_{1,t}}{1-\beta_1^t} \tag{15}$$

$$\hat{\boldsymbol{v}}_{2,t} = \frac{\boldsymbol{v}_{2,t}}{1-\beta_2^t} \tag{16}$$

$$\boldsymbol{W}_t = \boldsymbol{W}_{t-1} - \eta \frac{\hat{\boldsymbol{v}}_{1,t}}{\sqrt{\hat{\boldsymbol{v}}_{2,t}} + \varepsilon} \tag{17}$$

where η is the learning rate; β_1 and β_2 are the exponential decay rates for the first and the second moment estimates, respectively; subscript t indicates the time-step; β_1^t and β_2^t are the tth power of β_1 and β_2, respectively; J is the objective as a function of the

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vIFOndGsi5AAAAAA:CzOjbZGfofMEOE4IdIDVjkE6meT0oSjL7kQeQE7RMgFjvSYmOmuKG5gly5ahvRgSKx7g by Universita Di Pisa user on 28 November

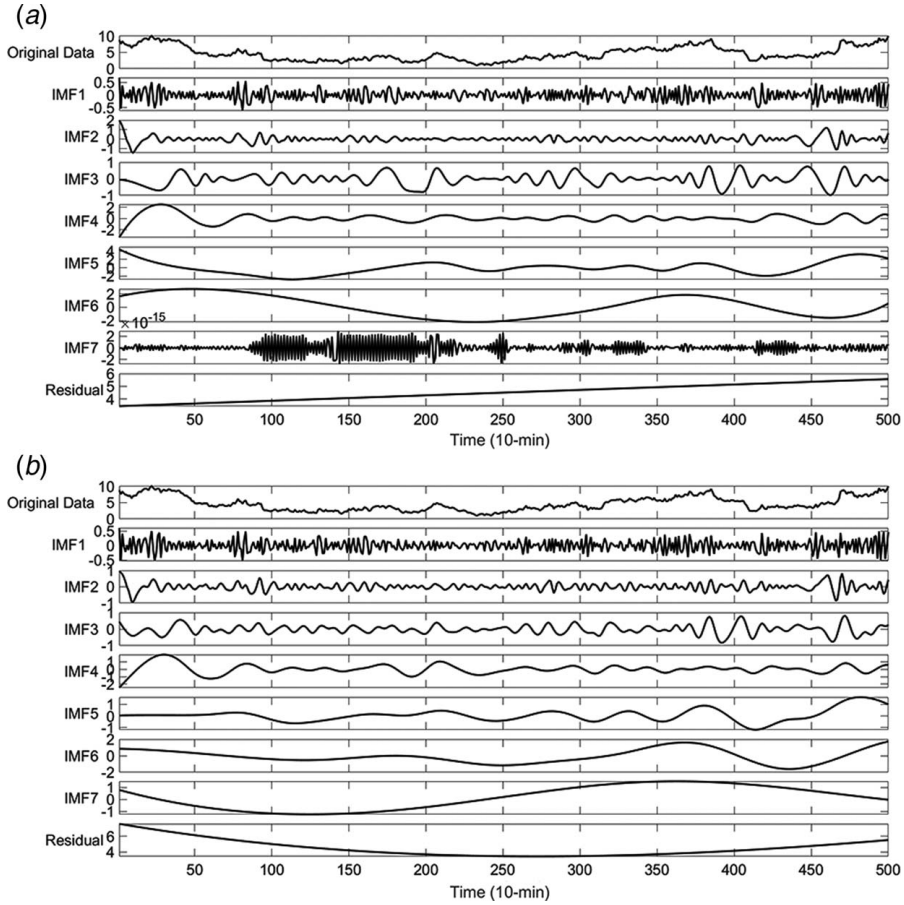


Fig. 6 A comparison of decomposed IMFs and the residual signal of 10-min mean wind speed data by (*a*) EMD and (*b*) EEMD. The demonstrated time series of wind data were collected from Feb. 7, 2017, 10:25 (UTC) to Feb. 8, 2017, 19:35 (UTC) at the studied site. Note that mixed modes clearly exist in the IMF6 obtained by EMD, but not in the IMF6 acquired by EEMD.

weight (i.e., the loss of the neural network); $\mathbf{G}$ is the gradient of the objective function; $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$ are the first and second moment vectors with initial values equal to zero, respectively; $\hat{\boldsymbol{v}}_1$ and $\hat{\boldsymbol{v}}_2$ are the estimates of the first and second moment vectors, respectively; and ε is a constant to avoid the denominator becoming zero. Here, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\varepsilon = 10^{-8}$ are used.

2.2 BO-LSTM Neural Networks. The accuracy of ANNs for a time series prediction, as well as other applications, is highly related to their architecture, which often includes the number of layers, the number of nodes in each layer, the connectivity pattern among layers, the choice of active functions, and the learning rate for the weight calculations. Here, four types of hyperparameters of LSTM neural networks, i.e., the learning rate, the number of hidden layers, the number of nodes in each hidden layer, and the time-steps that are used for recurrent weight calculations, are fine-tuned to minimize the loss function of the neural networks by applying a BO algorithm. It is worth noting that, except for the learning rate is continuous, the rest of three hyperparameters are all discrete variables. In order to address the nonlinear mixed discrete-continuous functions, many optimization algorithms have been extensively investigated in the mathematical and engineering literature [32], for example, the generalized benders decomposition (GBD) algorithm [33] and the extended cutting-plane (ECP) algorithm [34]. However, the global solution of the nonlinear mixed discrete-continuous problems cannot be guaranteed by the aforementioned two methods, because the feasible region is truncated during the linearization of the most violated constraints for ECP and by support functions for GBD [32]. An extensive survey of optimization algorithm to address the nonlinear mixed discrete-continuous problems is beyond the scope of this paper and readers may refer to the literature (e.g., Refs. [32–35]).

As a global optimization algorithm, BO is particularly appropriate when the objective function and constraints are unknown and computationally expensive to be evaluated [36,37]. The approach iteratively constructs a prior stochastic process (here, a Gaussian process (GP)) as the probabilistic surrogate model for a performance function, calculates a posterior process by conditioning on the previous performance evaluations, and searches for the most promising next design by maximizing a prescribed acquisition function. To handle the mixed discrete-continuous optimization problems herein, the continuous design variables obtained after each BO iteration are rounded toward the nearest integer for the three discrete hyperparameters of LSTM neural networks.

Let the four hyperparameters of the LSTM neural networks be represented by $\boldsymbol{\theta} = [\theta^1, \theta^2, \theta^3, \theta^4]$, where θ^1, θ^2, θ^3, and θ^4 represent the learning rate, the number of hidden layers, the number of nodes in each hidden layer, and the time-steps of the LSTM neural networks, respectively. There exists a relationship between the hyperparameters and the loss of the neural network described as $f(\boldsymbol{\theta})$. An optimization problem can be formulated as follows:

$$\begin{aligned} &\text{Find } \boldsymbol{\theta}^* = \arg\ \min f(\boldsymbol{\theta}) \\ &\text{Subject to } \boldsymbol{\theta}^L \le \boldsymbol{\theta} \le \boldsymbol{\theta}^H \end{aligned} \tag{18}$$

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vFOndGsI5AAAAA:CzOjbZGfofMEOE4IdIDVjkE6meT0oSJL7kQeQE7RMgFjvSYmOmuKG5gly5ahvRgSKx7g by Universita Di Pisa user on 28 November

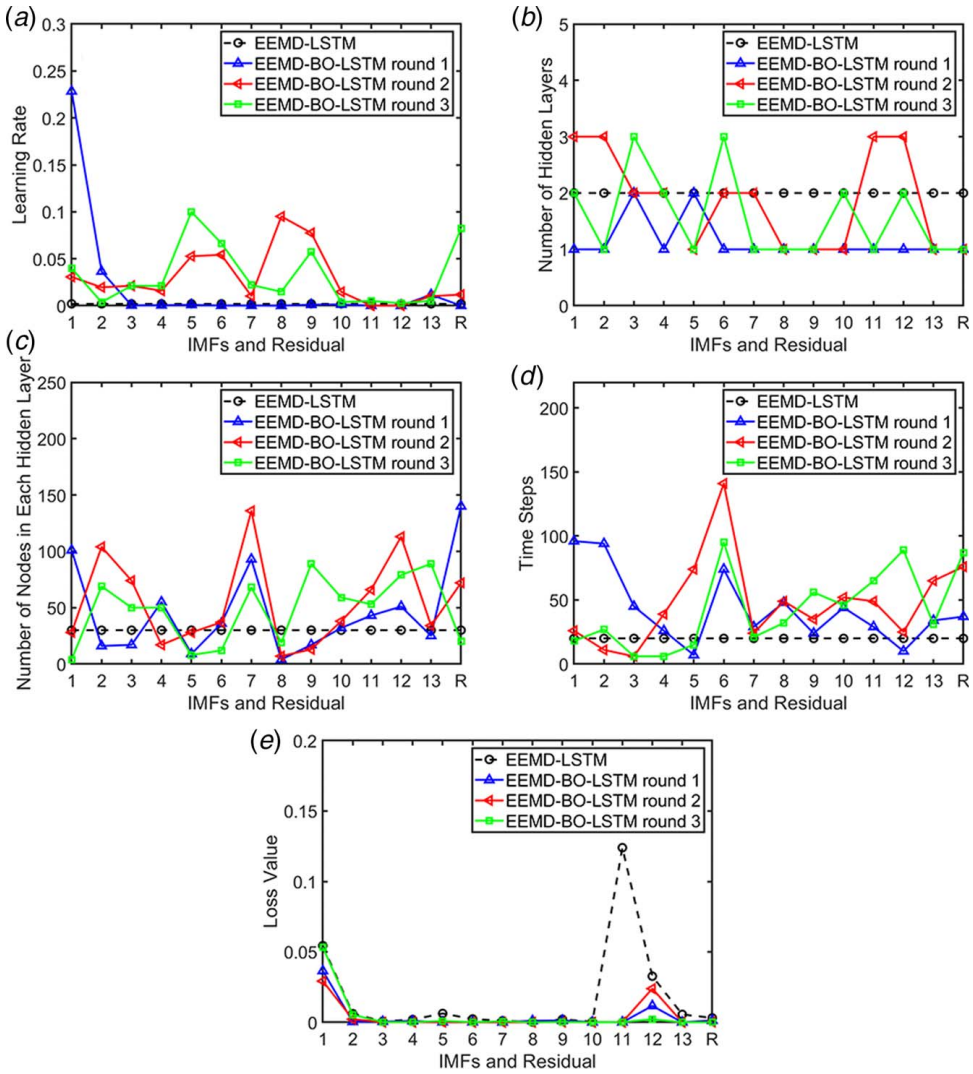


Fig. 7 A comparison of hyperparameters: (*a*) learning rate, (*b*) number of hidden layers, (c) number of nodes in each hidden layer, (*d*) time-steps, and (*e*) loss values of the initial LSTM neural networks and the BO-optimized LSTM neural networks for each EEMD-decomposed component in the case study of wind speed prediction

Table 1 The hyperparameters and the loss functions of the initial and optimized LSTM neural networks obtained by Bayesian optimization for the EEMD-decomposed components in the third round of rolling-origin-recalibration evaluation of the wind speed prediction case

	Initial LSTM neural network					Optimized LSTM neural network				
EEMD-decomposed component	θ^1	θ^2	θ^3	θ^4	L	θ^1	θ^2	θ^3	θ^4	L
IMF1	0.002000	2	30	20	0.054388	0.039780	2	4	18	0.053293
IMF2	0.002000	2	30	20	0.006086	0.003826	1	69	27	0.005373
IMF3	0.002000	2	30	20	0.000689	0.021240	3	50	6	0.000210
IMF4	0.002000	2	30	20	0.001915	0.021240	2	50	6	0.000147
IMF5	0.002000	2	30	20	0.006257	0.100000	1	8	15	0.000905
IMF6	0.002000	2	30	20	0.002334	0.066380	3	12	95	0.000097
IMF7	0.002000	2	30	20	0.001067	0.022110	1	68	21	0.000647
IMF8	0.002000	2	30	20	0.000251	0.014760	1	19	32	0.000039
IMF9	0.002000	2	30	20	0.002092	0.057450	1	89	56	0.000272
IMF10	0.002000	2	30	20	0.000404	0.003896	2	59	46	0.000017
IMF11	0.002000	2	30	20	0.123986	0.005059	1	53	65	0.000128
IMF12	0.002000	2	30	20	0.032715	0.002610	2	79	89	0.002162
IMF13	0.002000	2	30	20	0.005594	0.005301	1	89	31	0.000002
Residual	0.002000	2	30	20	0.003112	0.082280	1	20	87	0.000105

Note: θ^1 = the learning rate, θ^2 = the number of hidden layers, θ^3 = the number of nodes in each hidden layer, and θ^4 = the time-steps, L = the loss value.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vIFOndGsi5AAAAAA:CzOjbZGfofMEOE4IdIDVjkE6meTooSjL7kQeQE7RMgFjvSYmOmuKG5gjy5ahvRgSKx7g by Universita Di Pisa user on 28 November

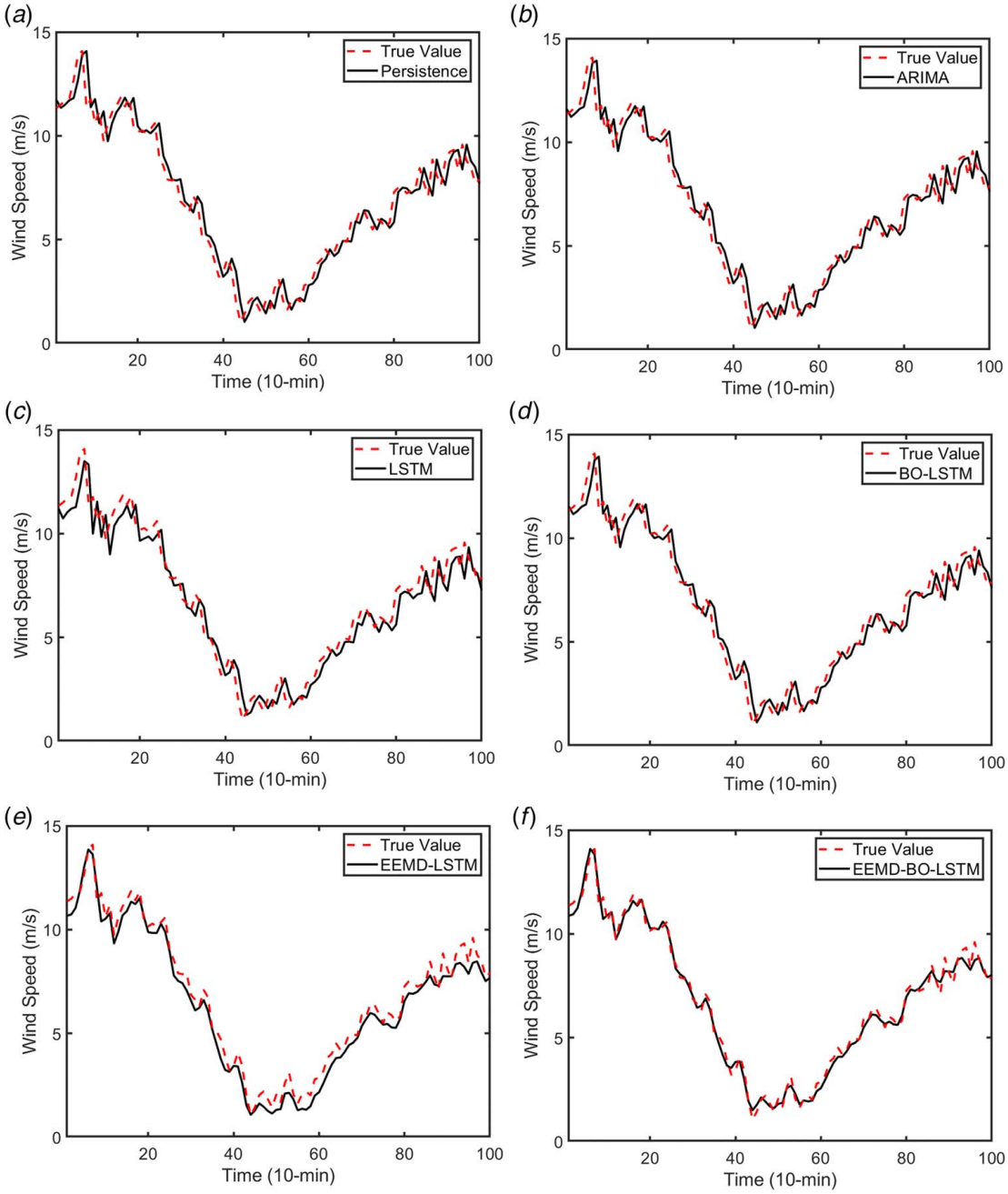


Fig. 8 Comparison of time series of the measured 10-min mean wind speed (true value) and the predicted ones by (*a*) the persistence model, (*b*) the ARIMA model, (*c*) the LSTM neural networks, (*d*) the BO-LSTM neural networks, (*e*) the EEMD-LSTM neural networks, and (*f*) the EEMD-BO-LSTM neural networks in the third round of rolling-origin-recalibration evaluation of the wind speed prediction case

To solve the above optimization problem, the objective function $f(\boldsymbol{\theta})$ is first represented by a probabilistic surrogate model using a Gaussian process. Let $f(\boldsymbol{\theta})$: $\chi \to \mathbb{R}$ be a Gaussian process, then any finite set of n samples $\{\boldsymbol{\theta}_i \in \chi\}_{i=1}^{n}$ induces a multivariate Gaussian distribution of $\mathbb{R}^n$, i.e.

$$\begin{bmatrix} f(\boldsymbol{\theta}_1) \\ \vdots \\ f(\boldsymbol{\theta}_n) \end{bmatrix} \sim N(\boldsymbol{\mu}, \mathbf{K}) \tag{19}$$

where $\boldsymbol{\mu} = [\mu_1, \mu_2, \ldots, \mu_n]^T$ is an $n \times 1$ mean vector and $\mathbf{K} = [k_{ij}(\boldsymbol{\theta}_i, \boldsymbol{\theta}_j)]_{n \times n}$ ($\boldsymbol{\theta}_i$ and $\boldsymbol{\theta}_j$ are two samples) is an $n \times n$ covariance matrix. $\boldsymbol{\mu}$ is determined by a mean function $\mathrm{M}(\boldsymbol{\theta})$ such that $\mu_i = \mathrm{M}(\boldsymbol{\theta}_i)$, where $\mathrm{M}(\boldsymbol{\theta})$ can be functions, e.g., zero, constant, linear, or other regression functions [38]. There exist different covariance functions (called kernels) that significantly enrich the distribution of varied Gaussian processes. The most commonly used covariance function is the automatic relevance determination (ARD) squared exponential kernel, which is expressed as

$$k_{SE}(\boldsymbol{\theta}_i, \boldsymbol{\theta}_j) = a_0 \exp\left\{-\frac{1}{2} r^2(\boldsymbol{\theta}_i, \boldsymbol{\theta}_j)\right\} \tag{20}$$

where α_0 is the covariance amplitude parameter, and

$$r^2(\boldsymbol{\theta}_i, \boldsymbol{\theta}_j) = \sum_{d=1}^{D} \frac{(\theta_{i,d} - \theta_{j,d})^2}{l_d} \tag{21}$$

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vIFOndGsi5AAAAAA:CzOjbZGfofMEOE4IdIDVjkE6meT0oSjL7kQeQE7RrMgFjvSYm0muKG5gly5ahvRgSKx7g by Universita Di Pisa user on 28 November

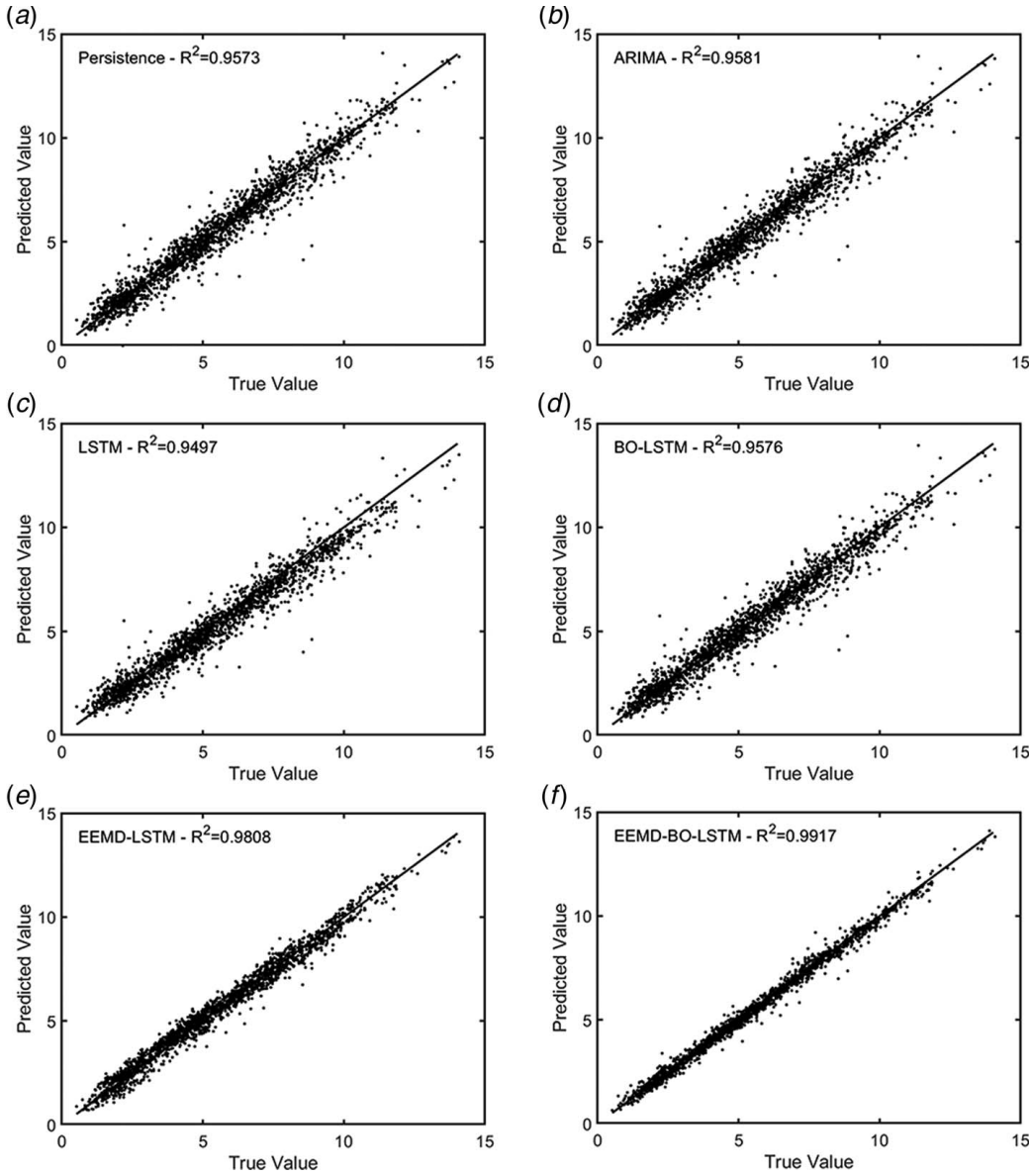


Fig. 9 Comparisons of the scatter plot of the measured 10-min mean wind speed (true value) and the predicted ones by (*a*) the persistence model, (*b*) the ARIMA model, (c) the LSTM neural networks, (*d*) the BO-LSTM neural networks, (*e*) the EEMD-LSTM neural networks, and (*f*) the EEMD-BO-LSTM neural networks in the third round of rolling-origin-recalibration evaluation of the wind speed prediction case

where $\theta_{i,d}$ and $\theta_{j,d}$ are the dth element in the $\boldsymbol{\theta}_i$ and $\boldsymbol{\theta}_j$, respectively; l_d is the length scale in the dth dimension; and D is the total number of dimensions of $f(\boldsymbol{\theta})$.

For practically optimization problems with a nonsmooth performance function, other kernels are often applied. For instance, the ARD Matern 3/2 kernel [39]

$$k_{Matern3/2}(\boldsymbol{\theta}_i, \boldsymbol{\theta}_j) = \alpha_0\left(1+\sqrt{3r^2(\boldsymbol{\theta}_i, \boldsymbol{\theta}_j)}\right)\exp\left\{-\sqrt{3r^2(\boldsymbol{\theta}_i, \boldsymbol{\theta}_j)}\right\} \tag{22}$$

and ARD Matern 5/2 kernel [39]

$$k_{Matern5/2}(\boldsymbol{\theta}_i, \boldsymbol{\theta}_j) = \alpha_0\left(1+\sqrt{5r^2(\boldsymbol{\theta}_i, \boldsymbol{\theta}_j)}+\frac{5}{3}r^2(\boldsymbol{\theta}_i, \boldsymbol{\theta}_j)\right) \times \exp\left\{-\sqrt{5r^2(\boldsymbol{\theta}_i, \boldsymbol{\theta}_j)}\right\} \tag{23}$$

Due to the latter two kernels being twice differentiable and capable of approximating the sample's function $f(\boldsymbol{\theta})$ well in nonsmooth regions, the ARD Matern 5/2 kernel is used in this study.

To avoid overfitting and to improve the numerical stability, a Gaussian noise with zero mean and variance σ^2 is added to $f(\boldsymbol{\theta})$. Hence, the prior distribution of the finite set of n samples drawn from $f(\boldsymbol{\theta})$ becomes

$$\begin{bmatrix} f(\boldsymbol{\theta}_1) \\ \vdots \\ f(\boldsymbol{\theta}_n) \end{bmatrix} \sim N(\boldsymbol{\mu}, \mathbf{K}+\sigma^2\mathbf{I}) \tag{24}$$

where $\mathbf{I}$ is an $n\times n$ identity matrix. Let the set of sampling data be $\mathbf{S_d} = [\mathbf{X}_n, \mathbf{Y}_n]$, where $\mathbf{X}_n = [\boldsymbol{\theta}_1, \boldsymbol{\theta}_2, \ldots, \boldsymbol{\theta}_n]^T$ and $\mathbf{Y}_n = [f(\boldsymbol{\theta}_1), f(\boldsymbol{\theta}_2), \ldots, f(\boldsymbol{\theta}_n)]^T$. Given the sampling data $\mathbf{S_d}$, the function value $f(\boldsymbol{\theta}_{n+1})$ at a new point $\boldsymbol{\theta}_{n+1}$ also follows a Gaussian distribution [38], which can be expressed as

$$f(\boldsymbol{\theta}_{n+1}) \sim N(\mu_{n+1|\mathbf{D}}, \sigma^2_{n+1|\mathbf{D}}) \tag{25}$$

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vIFOndGsI5AAAAAA:CzOjbZGfofMEOE4IdIDVjkE6meT0oSJL7kQeQE7RMgFjvSYm0muKG5gly5ahvRgSKx7g by Universita Di Pisa user on 28 November

Table 2 Results of the four accuracy metrics of the 10-min mean wind speed predicted by using different approaches

Prediction approach	RMSE (m/s)	MAE (m/s)	MAPE (%)	R^2
Persistence model	0.4753	0.3512	9.6634	0.9528
ARIMA model	0.4734	0.3501	9.7588	0.9532
LSTM neural networks	0.5162	0.3879	10.5640	0.9460
BO-LSTM neural networks	0.4225	0.3158	8.9568	0.9596
EEMD-LSTM neural networks	0.3164	0.2553	7.1491	0.9809
EEMD-BO-LSTM neural networks	0.2529	0.1989	5.6539	0.9855

Note: Each accuracy metric is the mean of accuracy values obtained by the three rounds of rolling-origin-recalibration evaluation of the wind speed prediction case.

The mean and variation of $f(\boldsymbol{\theta}_{n+1})$ can be calculated as

$$\mu_{n+1|\mathbf{D}} = m(\boldsymbol{\theta}_{n+1}) + k(\boldsymbol{\theta}_{n+1}, \mathbf{X}_n)(\mathbf{K} + \sigma^2\mathbf{I})^{-1}(\mathbf{Y}_n - \mathbf{m})$$
$$\sigma^2_{n+1|\mathbf{D}} = k(\boldsymbol{\theta}_{n+1}, \boldsymbol{\theta}_{n+1}) - k(\boldsymbol{\theta}_{n+1}, \mathbf{X}_n)(\mathbf{K} + \sigma^2\mathbf{I})^{-1}k(\mathbf{X}_n, \boldsymbol{\theta}_{n+1}) \quad (26)$$

where

$$\mathbf{m} = [m(\boldsymbol{\theta}_1), m(\boldsymbol{\theta}_2), \ldots, m(\boldsymbol{\theta}_n)]^T$$
$$k(\boldsymbol{\theta}_{n+1}, \mathbf{X}_n) = [k(\boldsymbol{\theta}_{n+1}, \boldsymbol{\theta}_1), k(\boldsymbol{\theta}_{n+1}, \boldsymbol{\theta}_2), \ldots, k(\boldsymbol{\theta}_{n+1}, \boldsymbol{\theta}_n)]$$
$$k(\mathbf{X}_n, \boldsymbol{\theta}_{n+1}) = k(\boldsymbol{\theta}_{n+1}, \mathbf{X}_n)^T$$
$$\mathbf{K} = \begin{bmatrix} k(\boldsymbol{\theta}_1, \boldsymbol{\theta}_1) & k(\boldsymbol{\theta}_1, \boldsymbol{\theta}_2) & \cdots & k(\boldsymbol{\theta}_1, \boldsymbol{\theta}_n) \\ k(\boldsymbol{\theta}_2, \boldsymbol{\theta}_1) & k(\boldsymbol{\theta}_2, \boldsymbol{\theta}_2) & \cdots & k(\boldsymbol{\theta}_2, \boldsymbol{\theta}_n) \\ \vdots & \vdots & \ddots & \vdots \\ k(\boldsymbol{\theta}_n, \boldsymbol{\theta}_1) & k(\boldsymbol{\theta}_n, \boldsymbol{\theta}_2) & \cdots & k(\boldsymbol{\theta}_n, \boldsymbol{\theta}_n) \end{bmatrix} \quad (27)$$

To calculate Eq. (26), we need to know D+3 parameters: α_0 and D length scales $l_1, l_2, \ldots, l_D$ for the covariance matrix $\mathbf{K}$; μ for the mean function $m(\boldsymbol{\theta})$; and σ for the Gaussian noise. In this paper, the D+3 parameters are determined by maximizing the likelihood of training data following the multivariate Gaussian distribution [38].

The next set of LSTM hyperparameters $\boldsymbol{\theta}_{n+1}$ is determined by maximizing an acquisition function that is denoted as $\boldsymbol{\theta}_{n+1} =$ Argmax $a(\boldsymbol{\theta}|[\mathbf{X}_n, \mathbf{Y}_n], \mathbf{p})$, where $\mathbf{X}_n$ and $\mathbf{Y}_n$ are the previous observations of the LSTM hyperparameters and the loss values, respectively, and $\mathbf{p}$ represents the parameters of the GP used in BO. Several popular acquisition functions exist, e.g., probability of improvement (PI), expected improvement (EI), and GP upper confidence bound (GP-UCB). Under the prior GP, these acquisition functions solely depend on the predictive mean function and the predictive variance function, which can be calculated by Eq. (26). Since the EI criterion has been shown to be better-behaved than the PI and does not require additional tuning parameters, such as the GP-UCB does [23], this paper uses the EI as the acquisition function, and it is expressed as [40]

$$a_{EI}(\boldsymbol{\theta}) = \begin{cases} (\mu(\boldsymbol{\theta}) - y_{best})C(z) + \sigma(\boldsymbol{\theta})P(z), & \text{if } \sigma(\boldsymbol{\theta}) > 0 \\ 0 & \text{if } \sigma(\boldsymbol{\theta}) = 0 \end{cases} \quad (28)$$

where P and C are the probability density function (PDF) and the cumulative distribution function (CDF) of the standard normal distribution, respectively; y_{best} is the current best observed loss value; and $z = (\mu(\boldsymbol{\theta}) - y_{best})/\sigma(\boldsymbol{\theta})$. Among all the obtained LSTM hyperparameters during the BO iterations, the one corresponding to the minimum loss function is selected as the optimal LSTM hyperparameter $\boldsymbol{\theta}^*$. A flowchart of the BO of the hyperparameters for the LSTM neural networks is provided in Fig. 2.

2.3 EEMD-BO-LSTM Neural Networks. Due to the variations in environmental conditions and other unforeseeable factors, a real-time series of complex systems often appears to be nonlinear and nonstationary, which causes difficulty when attempting to accurately predict the time series. One of the efficient methods to alleviate this prediction difficulty is to smartly decompose the original time series into a series of stable signals with regular modes that are easier to predict than the original time series. EEMD is such a means. As an improved method based on empirical mode decomposition (EMD), EEMD is an adaptive signal decomposition method that works by introducing white noise to the original signal to avoid the mode-mixing phenomenon of EMD [24].

Two assumptions are held to carry out EEMD on a time series $x(t)$. (1) The signal exists at least two extreme points including one maximum point and one minimum point. (2) The characteristic time scale of the signal is determined by the time interval of the two extreme points; based on which, the original signal can be decomposed into several IMFs. These IMFs should satisfy the following requirements: (1) In the entire data range, the number of local extreme points and zero crossings must be equal or differ by a maximum of one; (2) at any time, the mean value of the upper and lower envelopes determined by the local extreme points must be zero. The main steps to derive the IMFs and the residual signal of a time series $x(t)$ by EEMD are explained in detail below:

(1) Add a time series of white noise $W_o(t)$ with a mean of zero and a standard deviation of α to the original time series $x_o(t)$

$$x(t) = x_o(t) + W_o(t) \quad (29)$$

Table 3 The hyperparameters and the loss functions of the initial and optimized LSTM neural networks obtained by Bayesian optimization for the EEMD-decomposed components in the case study of wave height prediction

	Initial LSTM neural network					Optimized LSTM neural network				
EEMD-decomposed component	θ^1	θ^2	θ^3	θ^4	L	θ^1	θ^2	θ^3	θ^4	L
IMF1	0.002000	2	30	20	0.000442	0.056850	1	21	37	0.000283
IMF2	0.002000	2	30	20	0.001366	0.069120	3	15	12	0.000106
IMF3	0.002000	2	30	20	0.000312	0.100000	1	11	54	0.000066
IMF4	0.002000	2	30	20	0.001413	0.100000	1	51	40	0.000101
IMF5	0.002000	2	30	20	0.001230	0.014680	1	29	35	0.000053
IMF6	0.002000	2	30	20	0.000477	0.036574	2	22	19	0.000100
IMF7	0.002000	2	30	20	0.002972	0.029820	1	10	43	0.000243
IMF8	0.002000	2	30	20	0.001264	0.001000	1	23	1	0.000031
IMF9	0.002000	2	30	20	0.000318	0.040220	1	32	89	0.000003
Residual	0.002000	2	30	20	0.005716	0.055970	1	12	97	0.000007

Note: θ^1 = the learning rate, θ^2 = the number of hidden layers, θ^3 = the number of nodes in each hidden layer, and θ^4 = the time-steps, L = the loss value.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vIFOndGsi5AAAAAA:CzOjbZGfofMEOE4IdIDVjkE6meT0oSjL7kQeQE7RMgFjvSYm0muKG5gly5ahvRgSKx7g by Universita Di Pisa user on 28 November

(2) Connect all the local maxima of $x(t)$ by a cubic spline to form an upper envelop $U(t)$. Similarly, connect all the local minima of $x(t)$ by another cubic spline to form a lower envelop $L(t)$.

(3) Calculate the mean value $M(t)$ of the upper and lower envelops and then calculate the difference $H_o(t)$ between $x(t)$ and $M(t)$

$$H_o(t) = x(t) - M(t) \tag{30}$$

(4) Replace $x(t)$ by $H_o(t)$ and repeat steps 2 and 3 k times to obtain an IMF (defined as $c_1(t) = H_k(t)$) satisfying the following criterion:

$$\sum_{t=1}^{T} (H_{k-1}(t) - H_k(t))^2 \Big/ \sum_{t=1}^{T} H_{k-1}^2(t) < \beta \tag{31}$$

where T is the number of values in the time series and β is the threshold to end the iteration. Here, $\beta = 0.2$.

(5) Calculate the residual signal by subtracting the IMF obtained from step 4 from $x(t)$

$$r_o(t) = x(t) - c(t) \tag{32}$$

(6) Take $r_o(t)$ as the new original time series and repeat steps 2–5 to obtain m number of IMFs $c_{1,j}(t)$, $j = 1, \ldots, m$, where m is the number of IMFs and $r_1(t)$ corresponds to the added white noise.

(7) Update $x(t)$ by adding another white noise $W_o(t)$ with the same mean and standard deviation to the original signal $x_o(t)$.

(8) Repeat steps 2–7 n times to obtain n series of IMFs $c_{i,j}(t)$ and n number of $r_i(t)$, where $i = 1, \ldots, n$. Here, n is the ensemble number.

(9) Calculate the ensemble mean of IMFs $c_{i,j}(t)$ and the ensemble mean of $r_i(t)$ as the final IMFs and the final residual, respectively,

$$IMF_j^{EEMD}(t) = \frac{1}{n} \sum_{i=1}^{n} c_{i,j}(t) \tag{33}$$

$$r^{EEMD}(t) = \frac{1}{n} \sum_{i=1}^{n} r_i(t) \tag{34}$$

The number of IMFs m, the ensemble number n, and the amplitude of the white noise α play important roles in the EEMD process. The number of IMF m could be determined by different ways: (1) conduct the above step 6 until the residual signal is smaller than a prescribed threshold, (2) conduct the above step 6 until the residual signal becomes a monotonic function, or (3) predefine the number of IMFs as suggested by Wu and Huang (i.e., fix(log2(N))—1, where fix(x) rounds x toward zero and N is the number of records in the time series) [24], which is implemented in this paper. A large ensemble number could alleviate the influence of adding the white noise to the original signal and result in enough decomposed components with distinguished modes. However, it is computationally expensive when the ensemble number is too large. Here, the ensemble number $n = 80$. In this study, it is found that both too large and too small an amplitude of the standard deviation of the white noise causes a reduced accuracy of the hybrid method. Certain decomposed signals with small amplitudes could be buried when using too large an amplitude of the standard deviation of the white noise. On the other side, too small a standard deviation of the white noise may not distinguish signals with varied frequencies into different components. Here, the standard deviation of the white noise is selected to be 0.05. A thorough investigation of the selections of the number of IMFs, the ensemble number, and the amplitude of the white noise can be seen in Ref. [24].

The proposed time series prediction method based on the hybrid EEMD-BO-LSTM neural networks is therefore composed of three parts: (1) the decomposition of the original time series into components (i.e., IMFs and the residual signals) by EEMD, (2) the BO of the hyperparameters of the LSTM neural network for each decomposed component, and (3) the summation of each predicted component calculated by an optimal LSTM neural network, which is expressed as

$$\hat{y}^{(t+1)} = \sum_{j=1}^{m} F_j^{BO-LSTM}(IMF_j^{EEMD}(t)) + F_r^{BO-LSTM}(r^{EEMD}(t)) \tag{35}$$

where $F_j^{BO-LSTM}$ and $F_r^{BO-LSTM}$ represent the BO-LSTM neural networks for the jth IMF and the residual signal, respectively. A flowchart of the proposed time series prediction method is provided in Fig. 3.

3 Case Study

3.1 Wind Speed Prediction for Digital Twin of Wind Turbines. As suggested by Tao et al. [41], the wind turbine DT adopts a five-dimension architecture including the physical wind turbine, the virtual wind turbine, the service model, the DT data model, and the connection model (see Fig. 4). As a key component of the wind turbine DT, the service model plays an important role in the turbine's operation, e.g., turbine yaw and blade pitch control. To create the service model, it is necessary to develop short-term wind speed prediction models to optimize turbine yaw for improving wind energy capture and blade pitch control for alleviating turbine fatigue loads [17]. Additionally, due to the aerodynamic force scaling with the square of the wind speed, the wind speed prediction is of great importance to constructing a DT of other complex systems, for instance, the power forecast of wind farms [18,42], the structural load prediction of transmission towers [43], natural hazard evaluations (e.g., forest fire damage [44]), and aviation safety [45]. The full explanation of constructing the DT of these complex systems is beyond the scope of this paper and will be our future work.

In the first case study, the proposed hybrid EEMO-BO-LSTM neural networks approach is implemented for the time series prediction of 10-min mean wind speed. The raw data are 1-Hz wind speeds collected at a wind farm in a northern province in China, from Jan. 5, 2019, to Mar. 16, 2019. The time series of mean wind speeds of the raw wind data for every 10-min period are calculated and named as 10-min mean wind speeds, which are used to train the hybrid time series prediction method and to test its accuracy.

In addition to the proposed EEMD-BO-LSTM neural networks approach, five other models are tested for predicting the 10-min mean wind speed: the persistence model, ARIMA model, LSTM neural networks, BO-LSTM neural networks, and EEMD-LSTM neural networks. The persistence model takes the true value of the current time as the predicted value of the next time-step. In EEMD-LSTM neural networks and EEMD-BO-LSTM neural networks, 13 IMFs and a residual are determined according to Wu and Huang's criterion, as explained in Sec. 2.3. To have a fair comparison, the hyperparameters of the LSTM neural network and the EEMD-LSTM neural networks are the same. In addition, the parameters of the ARIMA model and the BO-LSTM model are fine-tuned by the Auto-ARIMA approach, which fits the best ARIMA model according to a provided information criterion [46], and the Bayesian optimization, respectively. Hence, the tunable parameters of both the ARIMA model and the BO-LSTM model are at near-optimal values. To measure the accuracy of the aforementioned prediction approaches, this paper uses four different metrics, the root mean squared error (RMSE), the mean absolute error (MAE), the mean absolute percentage error (MAPE), and the coefficient of determination (R^2), which are expressed as follows:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2} \tag{36}$$

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vFOndGsI5AAAAA:CzOjbZGfofMEOE4IdIDVjkE6meT0oSjL7kQeQE7RMgFjvSYmOmuKG5gly5ahvRgSKx7g by Universita Di Pisa user on 28 November

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i| \tag{37}$$

$$MAPE = \frac{1}{N}\sum_{i=1}^{N}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \times 100\% \tag{38}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2} \tag{39}$$

where y_i, $\hat{y}_i$, $\bar{y}$ are the observed value, the predicted value, and the mean of the observed values, respectively. N is the number of predicted values.

One of the cross-validation methods for time series prediction, the rolling-origin-recalibration evaluation approach [47,48], is applied to evaluate and compare the performance of different models. As is shown in Fig. 5, 10,000 samples are involved in the evaluation and each circle represents 1000 successive samples. In the first round of evaluation, the first 4000 samples are used for training and the next 2000 samples are used for testing; in the second round of evaluation, the first 6000 samples are used for training and the testing set are rolling to the next 2000 samples; in the third round of evaluation, the first 8000 samples are used for training and the testing set are rolling to the next 2000 samples. When training the prediction models, each training set is further split into a training subset and a validation subset to avoid the potential bias caused by using the whole training set for loss value calculation. Specifically, the mean loss value of the validation subset is used as the objective function in BO and the testing set is independently used for the accuracy metrics calculation. The average accuracy metric of the three rounds of evaluation is viewed as the accuracy metric for each model.

3.2 Wave Height Prediction for Digital Twin of Offshore Structures. To demonstrate other applications, the proposed hybrid EEMD-BO-LSTM neural networks approach is implemented for the time series prediction of hourly significant wave height, which is important to construct DT of offshore structures, for example, offshore oil and gas platforms [49].

A numerical hindcast ocean-wave model is used to produce the significant wave height data agreeing reasonably well with measurement [50]. The produced 1500 hourly significant wave height data are from Jan. 1, 2001, to Mar. 4, 2001, in an offshore site in the center of North Sea of the North Atlantic Ocean. The first 1000 samples are served as the training data and the remaining 500 samples are used as the testing data. Similarly, the aforementioned six time-series prediction models (including the proposed one) are tested, and the results of the four accuracy metrics (RMSE, MAE, MAPE, R^2) are compared and discussed in Sec. 4.

4 Results and Discussion

4.1 Wind Speed Prediction. As a demonstration, Fig. 6 shows the wind speed decomposition obtained by EMD and EEMD using 500 data points of the 10-mintue mean wind speed. Seven IMFs and one residual signal are obtained from both methods (according to Wu and Huang's criterion, as explained in Sec. 2.3). While the majority of the decomposed signals appear to be similar using the two methods, the IMF7 obtained by EMD has clear mixing modes with varied frequencies and amplitudes that are not present in the IMF7 obtained by EEMD.

When using all the 10-min mean wind speed data for EEMD, the number of IMFs is determined to be 13, per Wu and Huang's criterion. Figure 7 provides the hyperparameters and the loss values (i.e., the mean squared error (MSE) of the predicted decomposed components) of the initial LSTM neural networks and the BO-optimized LSTM neural networks for each decomposed component. The specific data of the hyperparameters and the loss values in the first, second, and third round of rolling-origin-recalibration evaluation are presented in Tables 5 and 6 and Table 1, respectively. In this case study, the searching ranges of the hyperparameters (learning rate, number of hidden layers, number of nodes in each hidden layer, and time-step) are 1e−5 to 1, 1 to 4, 1 to 150, and 1 to 150, respectively. As shown in Fig. 7, the changing trends of hyperparameters and loss value in each round of rolling-origin-recalibration evaluation are kept in consistence. In Fig. 7(*a*), the learning rates for IMFs of rounds 2 and 3 are significantly increased, which indicates using a large learning rate to iteratively calculate the weights of the LSTM neural networks (see Eq. (17)) improves the prediction accuracy in this case study. After BO, around 62% of the LSTM neural networks become less dense, as their number of hidden layers is reduced from 2 to 1 (Fig. 7(*b*)) while 14% of their number of hidden layers is increased from 2 to 3. The numbers of nodes in each hidden layer for the majority of IMFs are significantly increased indicating that a larger number of nodes in the LSTM neural network increases the prediction accuracy (Fig. 7(*c*)). In addition, it is interesting to find that the majority of the time-steps of the LSTM neural networks are significantly increased indicating that more previous time series information is used in the prediction (Fig. 7(*d*)).

Figure 7(*e*) shows a comparison of the loss values of the predicted IMFs and the residual signal between the EEMD-LSTM neural networks method and the EEMD-BO-LSTM neural networks method. Based on the loss values of the EEMD-LSTM neural networks method (dashed curve in Fig. 7(*e*)), it is found that the LSTM neural networks with the same architecture perform differently for different decomposed components. This indicates that indeed it is necessary to apply different ANN architectures for different decomposed time series components. Additionally, after implementing BO to fine-tune the hyperparameters of the LSTM neural networks, the loss values of the predicted components decrease to different degrees. For instance, the losses of decomposed components with relatively low frequencies, e.g., IMF11, show a significant reduction after the BO process. However, for the high-frequency components, e.g., IMF 1, the prediction accuracy of the LSTM neural networks is not significantly improved by the BO process. This finding indicates that BO performs better in improving the predictive accuracy of LSTM neural networks for a relatively stable and low-frequency time series in this studied case.

The true 10-min mean wind speeds and the predicted values using the persistence model, ARIMA model, LSTM neural networks, BO-LSTM neural networks, EEMD-LSTM neural networks, and EEMD-BO-LSTM neural networks methods in the first, the second, and the third round of rolling-origin-recalibration evaluation are demonstrated in Figs. 10 and 11, and Fig. 8, respectively. It is found that the predicted time series by the LSTM neural networks and the BO-LSTM neural networks clearly present a time delay from the true values, as is also shown in the results obtained by the persistence model (e.g., Figs. 8(*a*),8(*c*), and 8(*d*)). However, this time delay phenomenon is well alleviated in the predicted results by the EEMD-LSTM neural networks and the EEMD-BO-LSTM neural networks approaches (e.g., Figs. 8(*e*) and 8(*f*)). The scatter plots of the true values versus the predicted values acquired via the six methods clearly show that the EEMD-BO-LSTM neural networks approach presents the strongest predictability with the largest coefficient of determination (e.g., Figs. 9,12, and 13).

Table 2 summarizes the prediction accuracy results of the aforementioned six approaches for the case study of wind speed prediction. This table clearly shows that the proposed EEMD-BO-LSTM neural networks approach produces the smallest prediction error (RMSE = 0.2529 m/s, MAE = 0.1989 m/s, MAPE = 5.6539%, and $R^2 = 0.9855$), when compared with that of the other five prediction methods.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vIFOndGsI5AAAAAA:CzOjbZGfofMEOE4IdIDVjkE6meT0oSjL7kQeQE7RMgFjvSYmOmuKG5gly5ahvRgSKx7g by Universita Di Pisa user on 28 November

Table 4 Results of the four accuracy metrics of the hourly significant wave height predicted by using different approaches

Prediction approach	RMSE (m)	MAE (m)	MAPE (%)	R^2
Persistence model	0.0849	0.0578	3.2425	0.9913
ARIMA model	0.0399	0.0258	1.4305	0.9982
LSTM neural networks	0.2326	0.2079	14.2937	0.9344
BO-LSTM neural networks	0.0607	0.0407	2.1101	0.9954
EEMD-LSTM neural networks	0.1277	0.0983	6.6866	0.9802
EEMD-BO-LSTM neural networks	0.0273	0.0183	1.0559	0.9992

4.2 Wave Height Prediction. Table 3 provides the hyperparameters and the loss values of the initial LSTM neural networks and the BO-optimized LSTM neural networks for each decomposed component in the case study of wave height prediction. In this case study, the searching ranges of the hyperparameters (learning rate, number of hidden layers, number of nodes in each hidden layer, and time-step) are 1e-5 to 1, 1 to 4, 1 to 100, and 1 to 150, respectively. To save the page of the paper, the comparison figures of hyperparameters of the initial and the BO-optimized LSTM neural networks, the demonstrated time series of the true and the predicted wave height, and the scatter plots of the true and the predicted values are provided in Figs. 14–16, respectively, in the Appendix. As shown in Table 3 and Fig. 14, the learning rates for all of the IMFs (except IMF 8) and the time-steps for the majority of IMFs are significantly increased, while the number of hidden layers and the number of nodes in each layer for the majority of IMFs are reduced after BO. This finding indicates that a simplified LSTM architecture could still produce accurate predictions by utilizing more previous time-step information. As summarized in Table 4, the proposed EEMD-BO-LSTM neural networks approach produces the most accurate results (RMSE = 0.0273 m, MAE = 0.0183 m, MAPE = 1.0559%, and $R^2 = 0.9992$) when compared with those of the other five prediction methods.

5 Conclusions

The growing development of DT technologies brings about an emphasis on the accurate time series prediction of important parameters in DTs. This paper presents a hybrid time series prediction method based on EEMD, BO, and LSTM neural networks. In the proposed method, the original time series is first decomposed into several IMFs and a residual time series by EEMD. Each decomposed time series component is used to train an LSTM neural network. To find their optimal architecture, BO is implemented to fine-tune the hyperparameters of the LSTM neural networks. The time series is then predicted by accumulating the predicted components using the optimal LSTM neural networks.

In the first case study, the proposed EEMD-BO-LSTM neural networks approach is used to predict the 10-min mean wind speed along with other five prediction methods, the persistence model, ARIMA, LSTM neural networks, BO-LSTM neural networks, and EEMD-LSTM neural networks. The results show that the proposed EEMD-BO-LSTM neural networks approach can well alleviate the time delay issue and produce the best prediction with RMSE = 0.2529 m/s, MAE = 0.1989 m/s, MAPE = 5.6539%, and $R^2 = 0.9855$, when compared with the accuracy metrics of the other five methods. This accurate time series prediction method can be applied in constructing the DT of wind turbines for different applications, for instance, gust speed prediction, wind power forecasting, and performance and condition monitoring of wind turbines.

In the second case study, the proposed EEMD-BO-LSTM neural networks approach is used to predict the hourly significant wave height. Compared with the other five prediction methods, the proposed approach achieves the most accurate prediction with RMSE = 0.0273 m, MAE = 0.0183 m, MAPE = 1.0559%, and $R^2 = 0.9992$, which demonstrates the wide application of the proposed approach for time series prediction when constructing the DT of offshore structures.

Although accurate time series prediction is obtained using the proposed EEMD-BO-LSTM neural networks, the current prediction is limited to one-step-ahead time series prediction for constructing a DT. In addition, other key components when developing a DT, e.g., virtual model, data model, and connection model, have not been considered in this paper. Hence, future work may extend the proposed approach for multi-step ahead DT prediction of time series and the development of other DT components. Moreover, regarding the optimization of ANN for time series prediction, it is interesting to investigate other ANN optimization algorithms, for example, the neural architecture search method that applies the gradient information and the reinforcement strategy [51], as well as other nonlinear mixed discrete-continuous algorithms.

Acknowledgment

The authors strongly acknowledge the support from the Ministry of Science and Technology of China, the National Natural Science Foundation of China (NSFC), and the Fundamental Research Funds for the Central Universities. The authors also thank the three reviewers for their very insightful comments which help improve the paper quality.

Funding Data

- The work was supported by the National Key R&D Plan Project from the Ministry of Science and Technology of China (grant number 2019YFB1705202), the National Natural Science Foundation of China (NSFC) (grant numbers 51905475, 51935009), and the Fundamental Research Funds for the Central Universities (grant number 2019QNA4006).

Conflict of Interest

There are no conflicts of interest.

Data Availability Statement

The datasets generated and supporting the findings of this article are obtainable from the corresponding author upon reasonable request.

Nomenclature

- b = the vector of bias parameters in the artificial neural networks
- d = embedding dimension that is the number of past values taken into consideration to predict one future value
- k = the kernel used in BO
- l = length scale for the covariance matrix
- m = number of IMFs in the EMD and EEMD methods
- p = the parameters of the Gaussian process (GP) used in BO
- C = the cumulative distribution function of the standard normal distribution
- D = the total number of dimensions of the loss function in BO
- F = the loss function of the artificial neural networks
- G = the gradient of the objective function in the Adam optimization algorithm
- I = identity matrix

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vIFOndGsi5AAAAAA:CzOjbZGfofMEOE4IdIDVjkE6meT0oSjL7kQeQE7RMgFjvSYmOmuKG5gly5ahvRgSKx7g by Universita Di Pisa user on 28 November

J = the objective function of weights in the Adam optimization algorithm
P = the probability density function of the standard normal distribution
T = number of values in the time series
$\mathbf{K}$ = the covariance matrix in a Gaussian process
$\boldsymbol{W}$ = the weight matrix for different neural network's elements
$k_{Matern3/2}$ = the automatic relevance determination Matern 3/2 kernel
k_{SE} = the automatic relevance determination squared exponential kernel
y_{best} = the current best observed objective function value in BO
$\boldsymbol{b}_h$ = the vectors of bias parameters allowing each node to learn an offset for the hidden layer
$\boldsymbol{b}_y$ = the vectors of bias parameters allowing each node to learn an offset for the output layer
$\boldsymbol{X}_n$ = the sampling set of hyperparameters
$\boldsymbol{Y}_n$ = the sampling set of objective function value
$f^{(t)}$ = the value of the forget gate in the LSTM neural networks
$g^{(t)}$ = the value of an input node in the LSTM neural networks
$h^{(t)}$ = the value of a hidden node in the neural networks
$i^{(t)}$ = the value of the input gate in the LSTM neural networks
$o^{(t)}$ = the value of the output gate in the LSTM neural networks
$s^{(t)}$ = the value of the internal state in the LSTM neural networks
$x^{(t)}$ = the value of the input in the neural networks
$y^{(t)}$ = the value of the output label for training in the neural networks
$\hat{\mathbf{y}}^{(t)}$ = the value of the output in the neural networks
R^2 = coefficient of determination
$c(t)$ = IMFs in the EMD and EEMD methods
$f()$ = the objective function in Bayesian optimization
$r(t)$ = residual signal in the EMD and EEMD methods
$x(t)$ = the observations of a time series
AR = autoregressive
GP-UCB = Gaussian process upper confidence bound
$H(t)$ = the difference between x(t) and m(t) in the EMD and EEMD methods
IoT = internet of things
$M()$ = mean function
O&M = operation and maintenance
$S()$ = Sigmoid activation function
$T()$ = function to describe the relationship between past values and predicted value in a time series
$W_o(t)$ = a time series of white noise in the EEMD method
$\boldsymbol{Sd}$ = the set of sampling data in Bayesian optimization
α_0 = the covariance amplitude parameter
β = threshold of the modified Cauchy-type criterion
β_1 = exponential decay rate for the first moment estimate
β_2 = exponential decay rate for the second moment estimate
ε = the constant to avoid the divisor becoming zero in the Adam optimization algorithm
η = the learning rate of the artificial neural networks
θ = the hyperparameters of the LSTM neural networks
μ = the mean vector in a Gaussian process
v_1 = the first moment vector in the Adam optimization algorithm
v_2 = the second moment vector in the Adam optimization algorithm
$\hat{v}_1$ = the estimate of the first moment vector
$\hat{v}_2$ = the estimate of the second moment vector
σ = standard deviation
$\varphi()$ = tanh activation function

Appendix

Table 5 The hyperparameters and the loss functions of the initial and optimized LSTM neural networks obtained by Bayesian optimization for the EEMD-decomposed components in the first round of rolling-origin-recalibration evaluation of the wind speed prediction case

	Initial LSTM neural network					Optimized LSTM neural network				
EEMD-decomposed component	θ^1	θ^2	θ^3	θ^4	L	θ^1	θ^2	θ^3	θ^4	L
IMF1	0.002000	2	30	20	0.043825	0.015460	1	101	96	0.036516
IMF2	0.002000	2	30	20	0.003833	0.041750	1	16	94	0.000389
IMF3	0.002000	2	30	20	0.003626	0.040710	2	17	45	0.000473
IMF4	0.002000	2	30	20	0.001484	0.100000	1	55	26	0.000900
IMF5	0.002000	2	30	20	0.002463	0.086340	2	9	7	0.000128
IMF6	0.002000	2	30	20	0.006022	0.035500	1	36	74	0.000131
IMF7	0.002000	2	30	20	0.001103	0.016030	1	93	29	0.000023
IMF8	0.002000	2	30	20	0.00768	0.002054	1	4	48	0.001064
IMF9	0.002000	2	30	20	0.003336	0.059490	1	17	24	0.001283
IMF10	0.002000	2	30	20	0.000184	0.010870	1	32	44	0.000051
IMF11	0.002000	2	30	20	0.003051	0.016030	1	43	29	0.000026
IMF12	0.002000	2	30	20	0.093351	0.034640	1	51	10	0.011680
IMF13	0.002000	2	30	20	0.000021	0.039620	1	25	34	0.000000
Residual	0.002000	2	30	20	0.002839	0.016030	1	140	37	0.001261

Note: θ^1 = the learning rate, θ^2 = the number of hidden layers, θ^3 = the number of nodes in each hidden layer, and θ^4 = the time-steps, L = the loss value.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vIFOndGsI5AAAAAA:CzOjbZGfofMEOE4IdIDVjkE6meT0oSjL7kQeQE7RMgFjvSYm0muKG5gjy5ahvRgSKx7g by Universita Di Pisa user on 28 November

Table 6 The hyperparameters and the loss functions of the initial and optimized LSTM neural networks obtained by Bayesian optimization for the EEMD-decomposed components in the second round of rolling-origin-recalibration evaluation of the wind speed prediction case

	Initial LSTM neural network					Optimized LSTM neural network				
EEMD-decomposed component	θ^1	θ^2	θ^3	θ^4	L	θ^1	θ^2	θ^3	θ^4	L
IMF1	0.002000	2	30	20	0.028861	0.030660	3	28	26	0.029259
IMF2	0.002000	2	30	20	0.002173	0.019610	3	104	11	0.002169
IMF3	0.002000	2	30	20	0.000643	0.021240	2	74	6	0.000098
IMF4	0.002000	2	30	20	0.000693	0.015870	2	17	39	0.000102
IMF5	0.002000	2	30	20	0.001229	0.052690	1	28	74	0.000124
IMF6	0.002000	2	30	20	0.000553	0.054220	2	37	141	0.000030
IMF7	0.002000	2	30	20	0.000012	0.010320	2	136	24	0.000034
IMF8	0.002000	2	30	20	0.005214	0.095140	1	7	49	0.000032
IMF9	0.002000	2	30	20	0.006486	0.077460	1	13	35	0.000191
IMF10	0.002000	2	30	20	0.000047	0.014680	1	38	52	0.000006
IMF11	0.002000	2	30	20	0.003404	0.000100	3	66	49	0.000074
IMF12	0.002000	2	30	20	0.013149	0.000204	3	113	25	0.023921
IMF13	0.002000	2	30	20	0.000120	0.010130	1	34	65	0.000000
Residual	0.002000	2	30	20	0.001729	0.011750	1	72	76	0.000231

Note: θ^1 = the learning rate, θ^2 = the number of hidden layers, θ^3 = the number of nodes in each hidden layer, and θ^4 = the time-steps, L = the loss value.

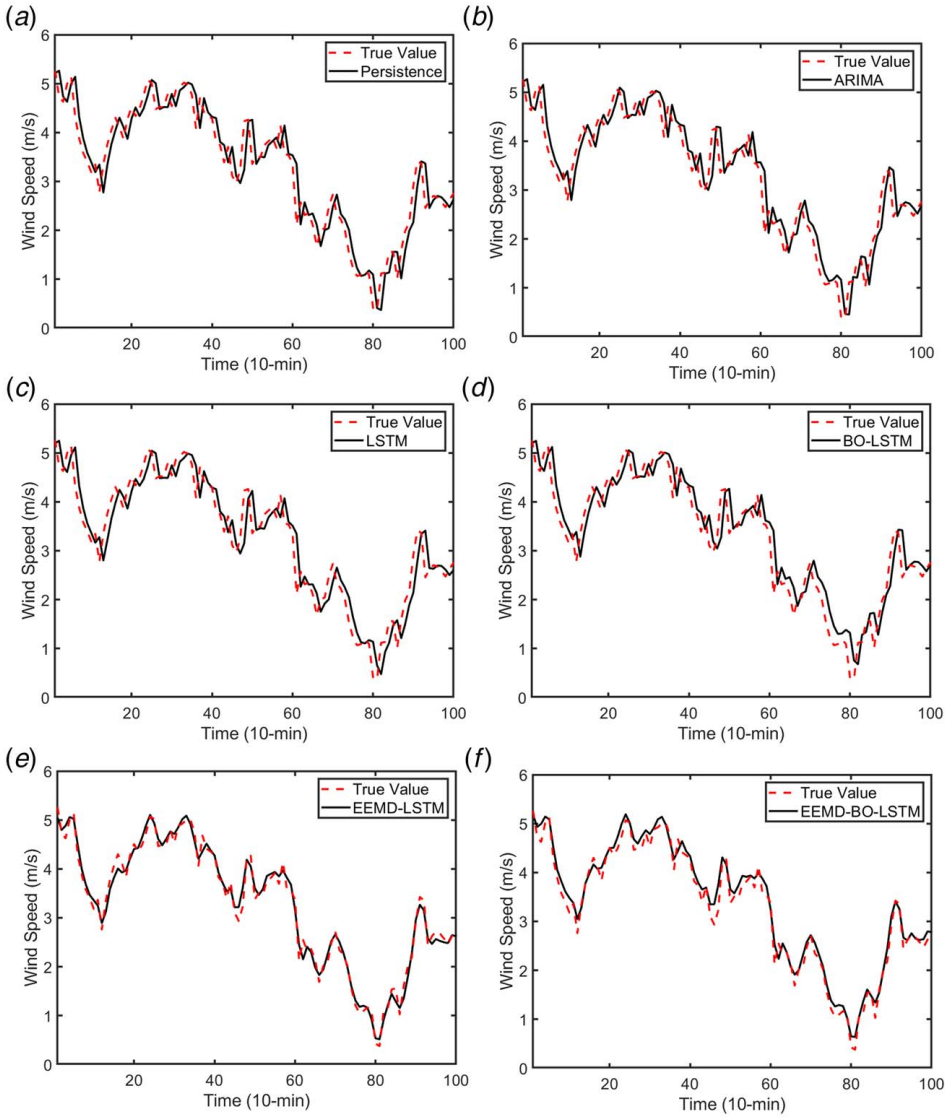


Fig. 10 Comparison of time series of the measured 10-min mean wind speed (true value) and the predicted ones by (*a*) the persistence model, (*b*) the ARIMA model, (c) the LSTM neural networks, (*d*) the BO-LSTM neural networks, (e) the EEMD-LSTM neural networks, and (*f*) the EEMD-BO-LSTM neural networks in the first round of rolling-origin-recalibration evaluation of the wind speed prediction case

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vIFOndGsi5AAAAAA:CzOjbZGfofMEOE4IdIDVjkE6meT0oSJL7kQeQE7RMgFjvSYm0muKG5gJy5ahvRgSKx7g by Universita Di Pisa user on 28 November

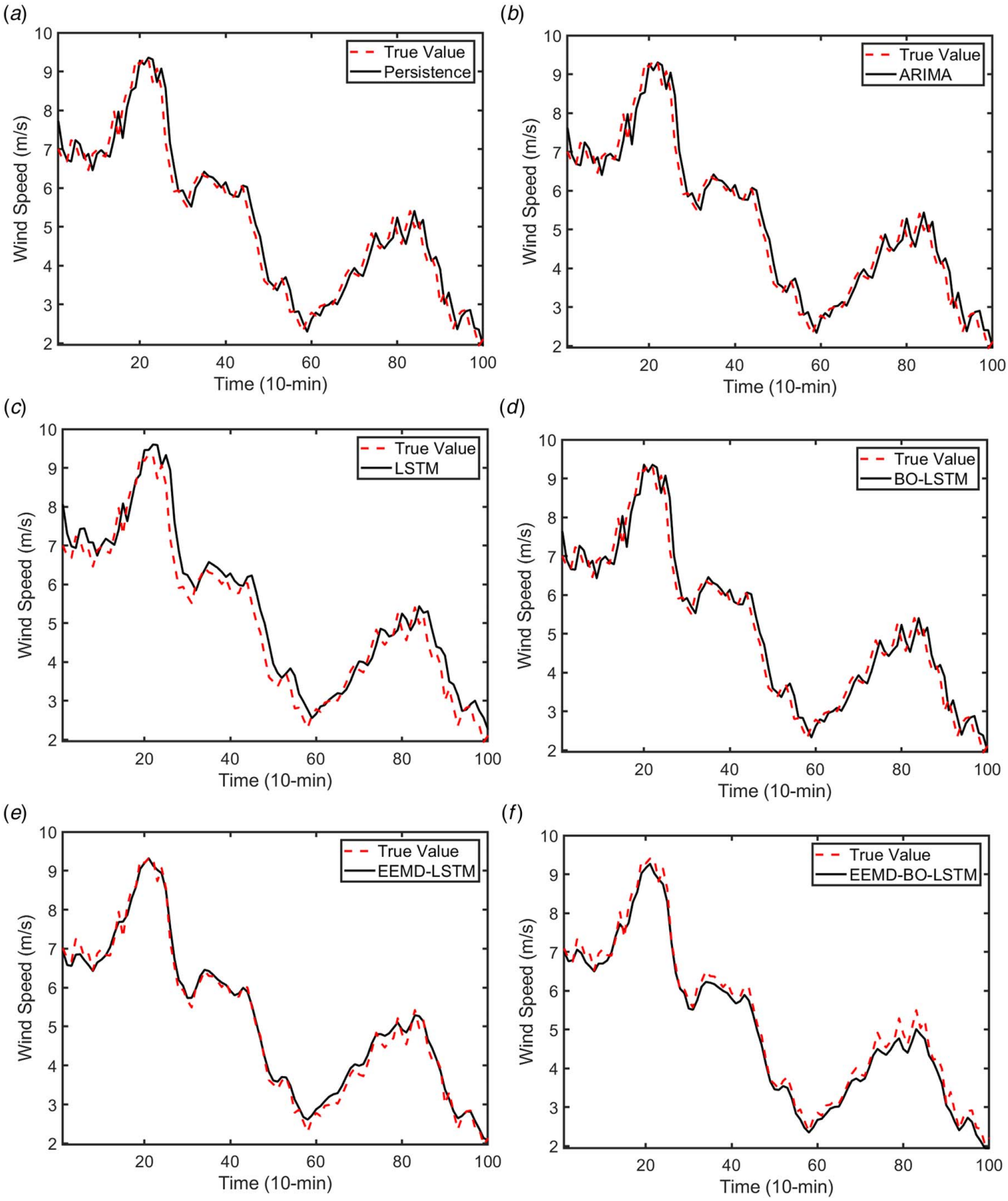


Fig. 11 Comparison of time series of the measured 10-min mean wind speed (true value) and the predicted ones by (*a*) the persistence model, (*b*) the ARIMA model, (*c*) the LSTM neural networks, (*d*) the BO-LSTM neural networks, (*e*) the EEMD-LSTM neural networks, and (*f*) the EEMD-BO-LSTM neural networks in the second round of rolling-origin-recalibration evaluation of the wind speed prediction case

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vIFOndGsi5AAAAAA:CzOjbZGfofMEOE4IdIDVjkE6meT0oSjL7kQeQE7RrMgFjvSYm0muKG5gJy5ahvRgSKx7g by Universita Di Pisa user on 28 November

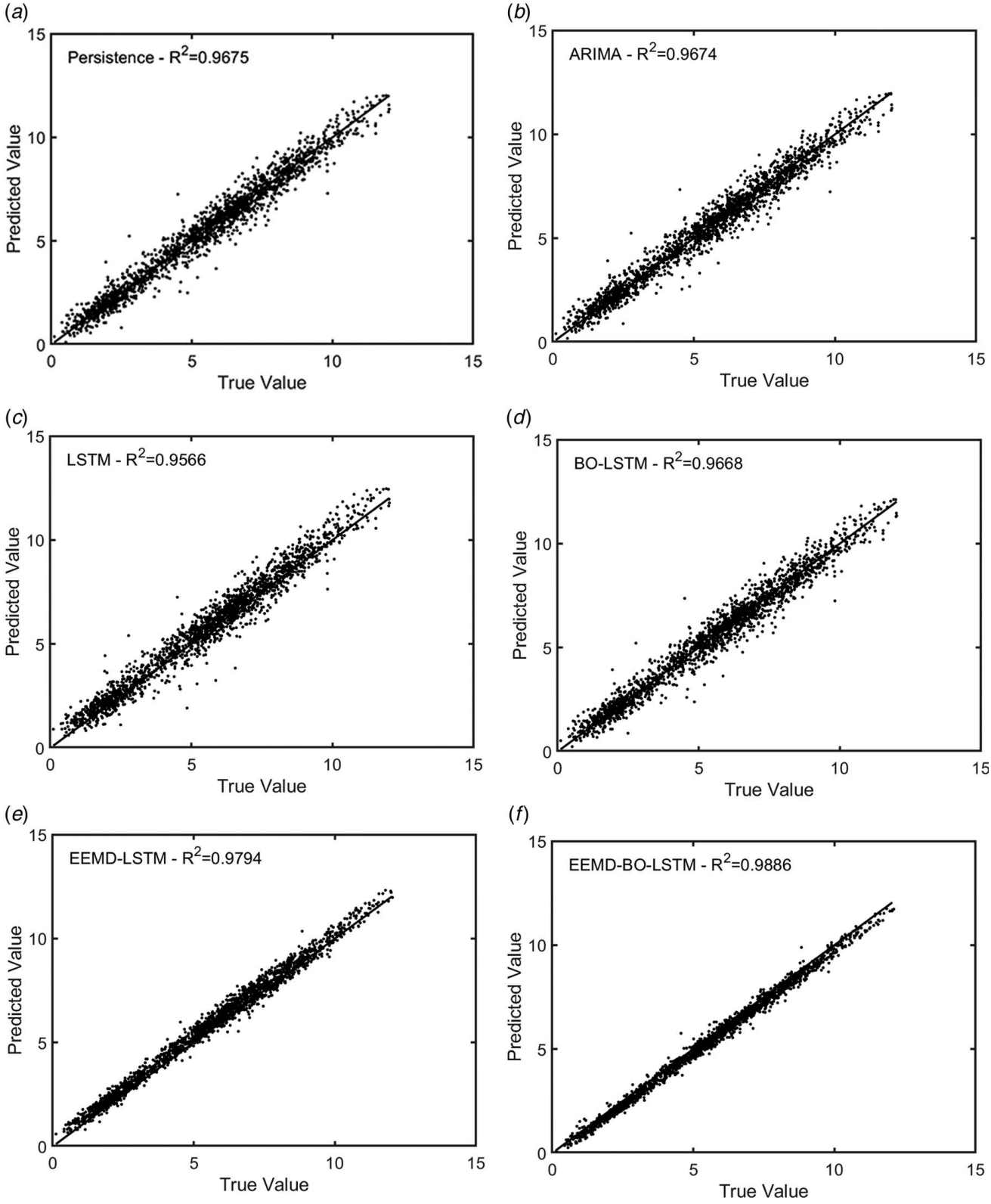


Fig. 12 Comparisons of the scatter plot of the measured 10-min mean wind speed (true value) and the predicted ones by (*a*) the persistence model, (*b*) the ARIMA model, (*c*) the LSTM neural networks, (*d*) the BO-LSTM neural networks, (*e*) the EEMD-LSTM neural networks, and (*f*) the EEMD-BO-LSTM neural networks in the first round of rolling-origin-recalibration evaluation of the wind speed prediction case

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vIFOndGsi5AAAAAA:CzOjbZGfofMEOE4IdIDVjkE6meT0oSjL7kQeQE7RrMgFjvSYm0muKG5gjy5ahvRgSKx7g by Universita Di Pisa user on 28 November

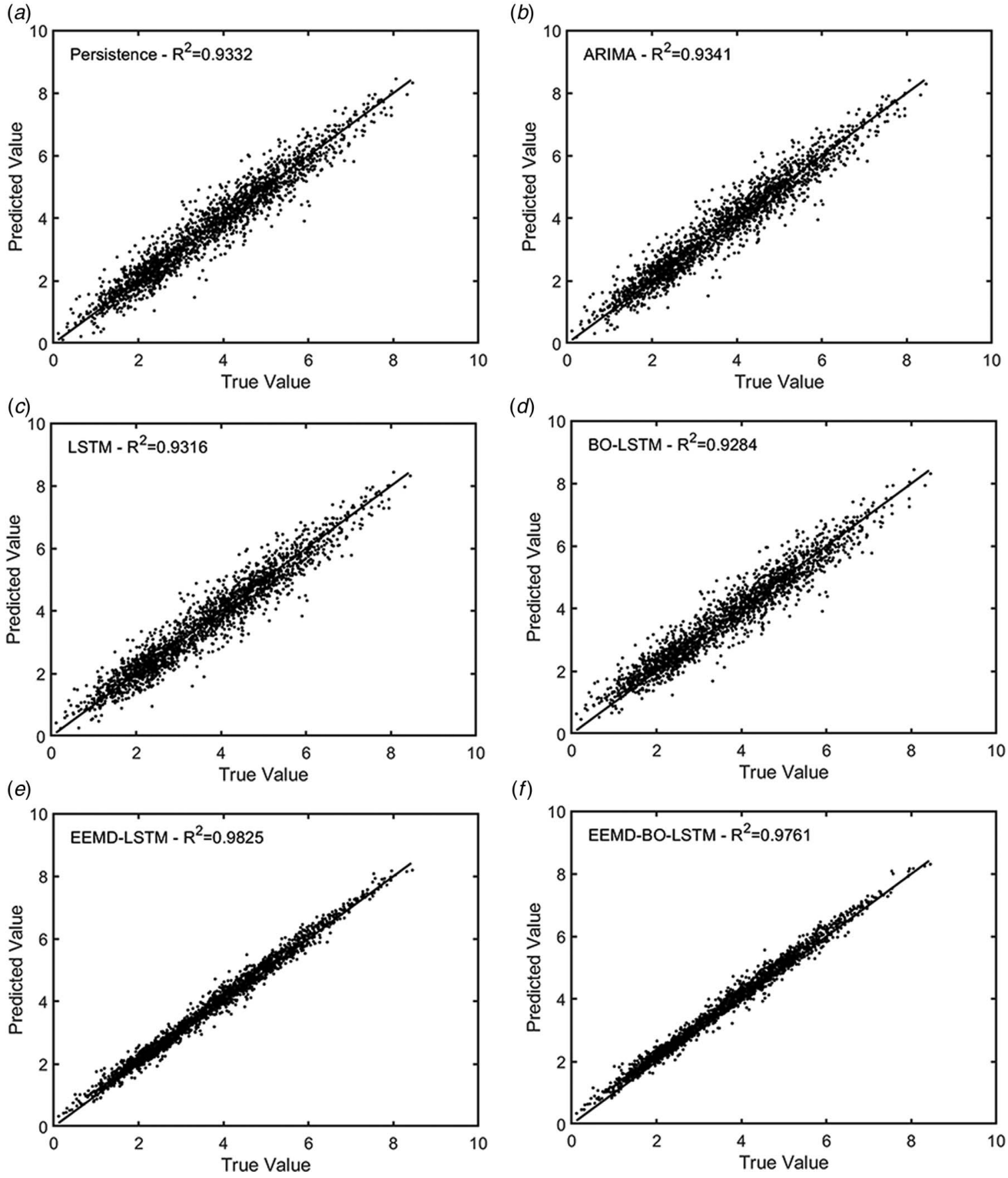


Fig. 13 Comparisons of the scatter plot of the measured 10-min mean wind speed (true value) and the predicted ones by (*a*) the persistence model, (*b*) the ARIMA model, (*c*) the LSTM neural networks, (*d*) the BO-LSTM neural networks, (*e*) the EEMD-LSTM neural networks, and (*f*) the EEMD-BO-LSTM neural networks in the second round of rolling-origin-recalibration evaluation of the wind speed prediction case

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vIFOndGsI5AAAAAA:CzOjbZGfofMEOE4IdIDVjkE6meT0oSjL7kQeQE7RrMgFjvSYm0muKG5gly5ahvRgSKx7g by Universita Di Pisa user on 28 November

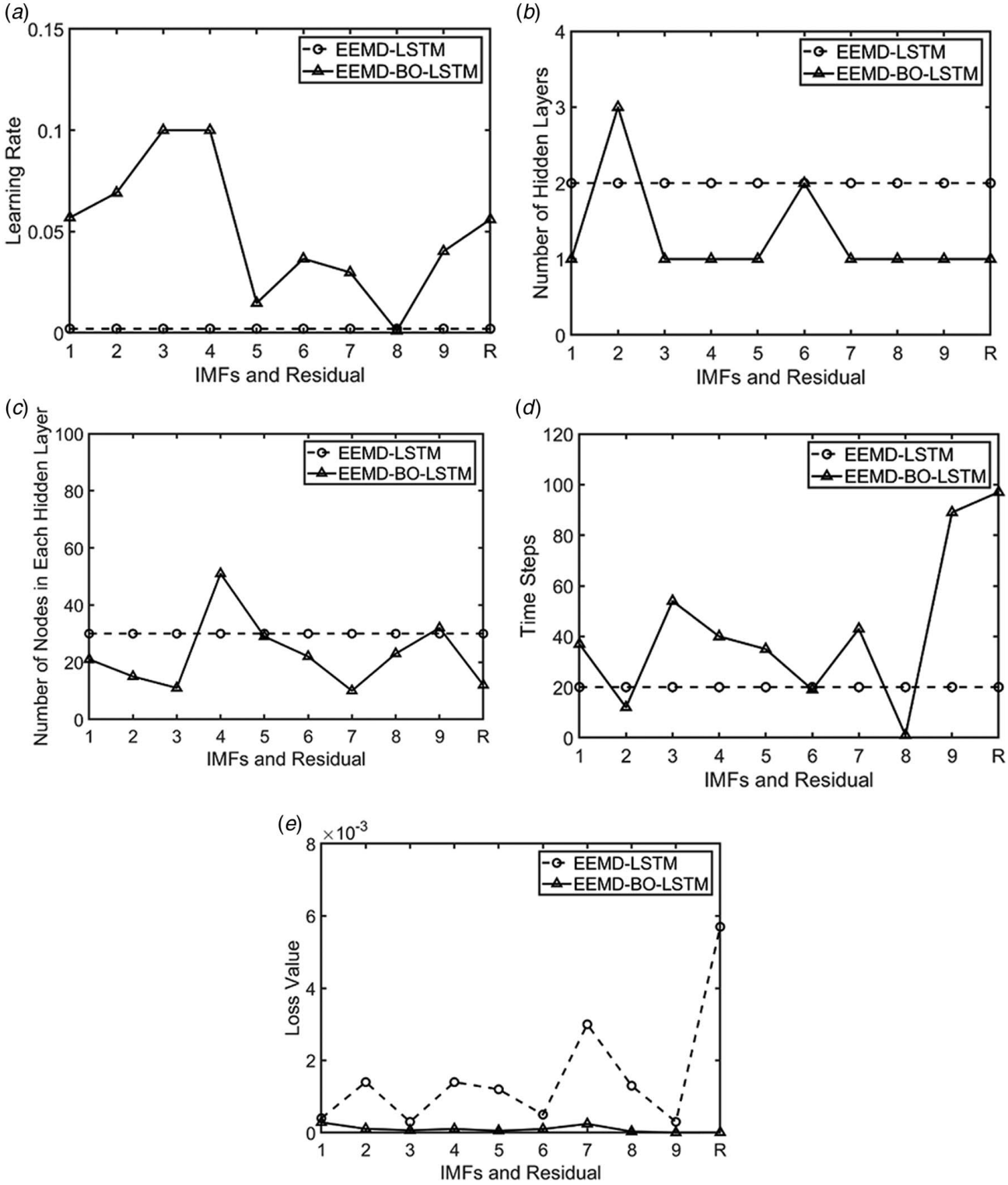


Fig. 14 A comparison of hyperparameters: (*a*) learning rate, (*b*) number of hidden layers, (*c*) number of nodes in each hidden layer, (*d*) time-steps, and (*e*) loss values of the initial LSTM neural networks and the BO-optimized LSTM neural networks for each EEMD-decomposed component in the case study of wave height prediction

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vIFOndGsi5AAAAAA:CzOjbZGfofMEOE4IdIDVjkE6meT0oSjL7kQeQE7RrMgFjvSYm0muKG5gly5ahvRgSKx7g by Universita Di Pisa user on 28 November

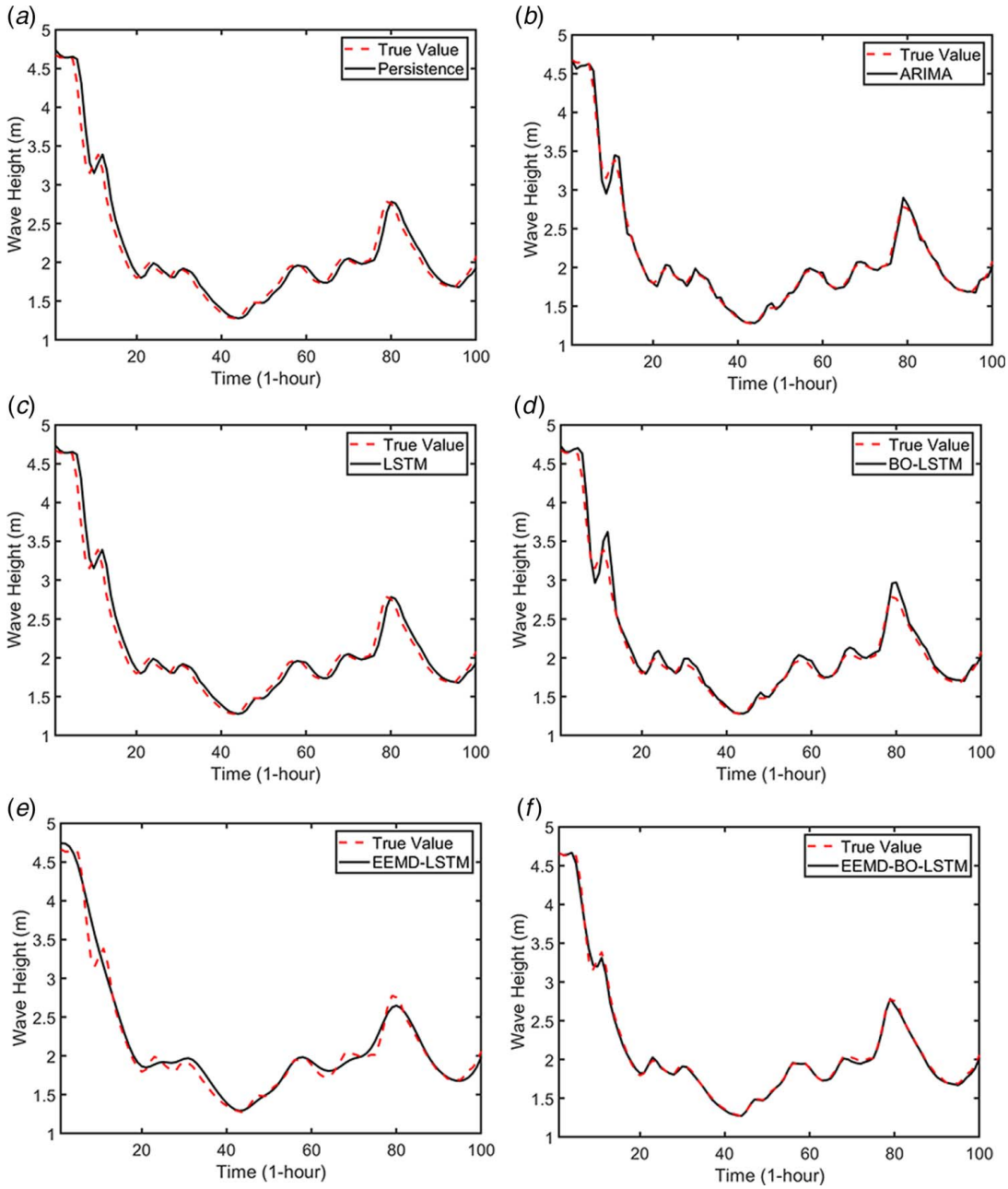


Fig. 15 Comparison of time series of the original hourly significant wave height (true value) and the predicted ones by (*a*) the persistence model, (*b*) the ARIMA model, (*c*) the LSTM neural networks, (*d*) the BO-LSTM neural networks, (*e*) the EEMD-LSTM neural networks, and (*f*) the EEMD-BO-LSTM neural networks in the case study of wave height prediction

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vIFOndGsI5AAAAAA:CzOjbZGfofMEOE4IdIDVjkE6meT0oSjL7kQeQE7RMgFjvSYmOmuKG5gIyI5ahvRgSKx7g by Universita Di Pisa user on 28 November

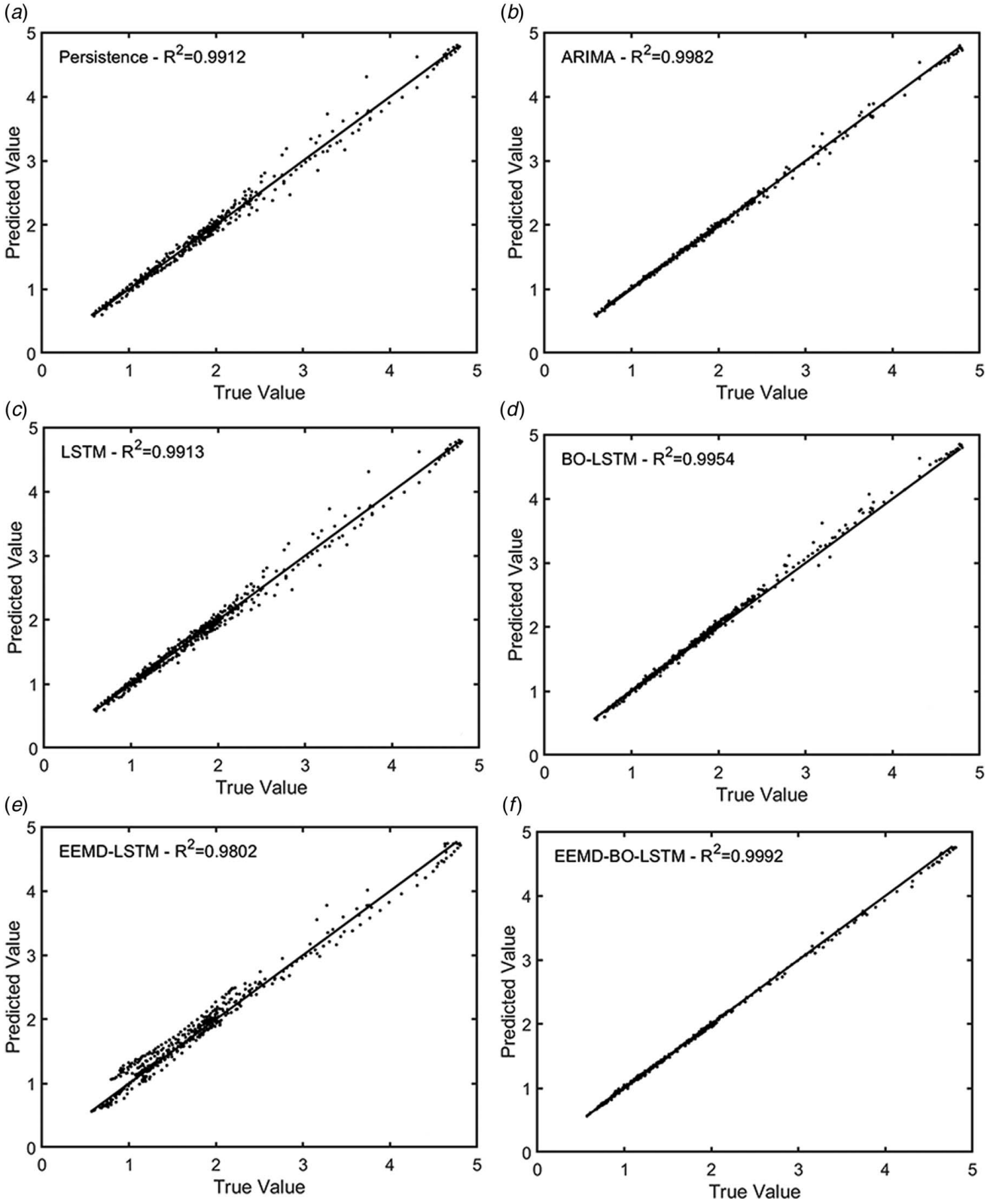


Fig. 16 Comparisons of the scatter plot of the original hourly significant wave height (true value) and the predicted ones by (*a*) the persistence model, (*b*) the ARIMA model, (c) the LSTM neural networks, (*d*) the BO-LSTM neural networks, (*e*) the EEMD-LSTM neural networks, and (*f*) the EEMD-BO-LSTM neural networks in the case study of wave height prediction

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vIFOndGsI5AAAAAA:CzOjbZGfofMEOE4IdIDVjkE6meT0oSjL7kQeQE7RrMgFjvSYm0muKG5gIy5ahvRgSKx7g by Universita Di Pisa user on 28 November

References

[1] Grieves, M., 2005, "Product Lifecycle Management: The New Paradigm for Enterprises," Int. J. Prod. Dev., **2**(1/2), pp. 71–84.
[2] Miller, A. M., Alvarez, R., and Hartman, N., 2018, "Towards an Extended Model-Based Definition for the Digital Twin," Comput.-Aided Des. Appl., **15**(6), pp. 880–891.
[3] Glaessgen, E. H., and Stargel, D. S., 2012, "The Digital Twin Paradigm for Future NASA and U.S. Air Force Vehicles," 53rd AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics and Materials Conference: Special Session on the Digital Twin, Honolulu, HI, Apr. 23–26, pp. 1–14.
[4] Tao, F., Sui, F., Liu, A., Qi, Q., Zhang, M., Song, B., Guo, Z., Lu, S. C.-Y., and Nee, A. Y. C., 2019, "Digital Twin-Driven Product Design Framework," Int. J. Prod. Res., **57**(12), pp. 3935–3953.
[5] Schleich, B., Anwer, N., Mathieu, L., and Wartzack, S., 2017, "Shaping the Digital Twin for Design and Production Engineering," CIRP Ann.—Manuf. Technol., **66**(1), pp. 141–144.
[6] Tuegel, E. J., Ingraffea, A. R., Eason, T. G., and Spottswood, S. M., 2011, "Reengineering Aircraft Structural Life Prediction Using a Digital Twin," Int. J. Aerosp. Eng., **2011**, pp. 1–14.
[7] Boschert, S., Heinrich, C., and Rosen, R., 2018, "Next Generation Digital Twin," *Proc. TMCE 2018*, I. Horváth, J. P. S. Rivero, and P. M. H. Castellano, eds., Organizing Committee of TMCE 2018, Las Palmas de Gran Canaria, Spain, pp. 209–217.
[8] Parrot, A., and Warshaw, L., 2017, *Industry 4.0 and the Digital Twin—Manufacturing Meets its Match*, Deloitte University Press, New York.
[9] Grieves, M., Vickers, J., et al.et al., 2017, "Digital Twin: Mitigating Unpredictable Undesirable Emergent Behavior in Complex Systems," *Transdisciplinary Perspective on Complex Systems*, F.-J. Kahlen, ed., Springer International Publishing, Switzerland.
[10] Zhuang, C., Liu, J., and Xiong, H., 2018, "Digital Twin-Based Smart Production Management and Control Framework for the Complex Product Assembly Shop-Floor," Int. J. Adv. Manuf. Technol., **96**(1–4), pp. 1149–1163.
[11] Zhang, H., LIu, Q., Chen, X., Zhang, D., and Leng, J., 2017, "A Digital Twin-Based Approach for Designing and Mulit-Objective Optimization of Hollow Glass Production Line," IEEE Access, **5**, pp. 26901–26911.
[12] Malik, A. A., and Bilberg, A., 2018, "Digital Twins of Human Robot Collaboration in a Production Setting," Procedia Manuf., **17**, pp. 278–285.
[13] Tao, F., Cheng, J., Qi, Q., Zhang, M., Zhang, H., and Sui, F., 2018, "Digital Twin-Driven Product Design, Manufacturing and Service with big Data," Int. J. Adv. Manuf. Technol., **94**(9–12), pp. 3563–3576.
[14] Marugán, A. P., Márquez, F. P. G., Perez, J. M. P., and Ruiz-Hernández, D., 2018, "A Survey of Artificial Neural Network in Wind Energy Systems," Appl. Energy, **228**, pp. 1822–1836.
[15] Doucoure, B., Agbossou, K., and Cardenas, A., 2016, "Time Series Prediction Using Artificial Wavelet Neural Network and Multi-Resolution Analysis: Application to Wind Speed Data," Renewable Energy, **92**, pp. 202–211.
[16] Ma, L., Luan, S., Jiang, C., Liu, H., and Zhang, Y., 2009, "A Review on the Forecasting of Wind Speed and Generated Power," Renewable Sustainable Energy Rev., **13**(4), pp. 915–920.
[17] Wang, Z., Zhang, X., Huang, H.-Z., and Mourelatos, Z. P., 2016, "A Simulation Method to Estimate Two Types of Time-Varying Failure Rate of Dynamic Systems," ASME J. Mech. Des., **138**(12), p. 121404.
[18] Kulkarni, P. A., Hu, W., Dhoble, A. S., and Padole, P. M., 2017, "Statistical Wind Prediction and Fatigue Analysis for Horizontal-Axis Wind Turbine Composite Material Blade Under Dynamic Loads," Adv. Mech. Eng., **9**(9), pp. 1–26.
[19] Ma, J., and Kim, H. M., 2016, "Predictive Model Selection for Forecasting Product Returns," ASME J. Mech. Des., **138**(5), p. 054501.
[20] Lukoseviciute, K., and Ragulskis, M., 2010, "Evolutionary Algorithms for the Selection of Time Lags for Time Series Forecasting by Fuzzy Inference Systems," Neurocomputing, **73**(10–12), pp. 2077–2088.
[21] Wurdinger, K., 2009, *Investigating an Evolutionary Strategy to Forecast Time Series*, Universiteit Leiden Opleiding Informatica, Leiden, Netherlands.
[22] Cadenas, E., and Rivera, W., 2010, "Wind Speed Forecasting in Three Different Regions of Mexico, Using a Hybrid ARIMA—ANN Model," Renewable Energy, **35**(12), pp. 2732–2738.
[23] Snoek, J., Larochelle, H., and Adams, R. P., 2012, "Practical Bayesian Optimization of Machine Learning Algorithms," Proceedings of the 25th International Conference on Neural Information Processing Systems, Lake Tahoe, NV, Dec. 3–6, pp. 2951–2959.
[24] Wu, Z., and Huang, N. E., 2009, "Ensemble Empirical Mode Decomposition: A Noise-Assisted Data Analysis Method," Adv. Adapt. Data Anal., **1**(1), pp. 1–41.
[25] Taieb, S. B., Sorjama, A., and Bontempi, G., 2010, "Multiple-Output Modeling for Multi-Step-Ahead Time Series Forecasting," Neurocomputing, **73**(10–12), pp. 1950–1957.
[26] Lipton, Z. C., Berkowitz, J., and Elkan, C., 2015, "A Critical Review of Recurrent Neural Networks for Sequence Learning," arXiv:1506.00019v4 [cs.LG].
[27] Bengio, Y., Simard, P., and Frasconi, P., 1994, "Learning Long-Term Dependencies With Gradient Descent is Difficult," IEEE Trans. Neural Networks, **5**(2), pp. 157–166.
[28] Hochreiter, S., and Schmidhuber, J., 1996, "Bridging Long Time Lags by Weight and Guessing and "Long Short-Term Memory", Spatiotemporal Models Biological Artificial Systems, **37**, pp. 65–72.
[29] Hochreiter, S., and Schmidhuber, J., 1997, "Long Short-Term Memory," Neural Comput., **9**(8), pp. 1735–1780.
[30] Gers, F. A., Schmidhuber, J., and Cummins, F., 2000, "Learning to Forget: Continual Prediction With LSTM," Neural Comput., **12**(10), pp. 2451–2471.
[31] Kingma, D. P., and Ba, J., 2014, "Adam: A Method for Stochastic Optimization," arXiv preprint arXiv:1412.6980.
[32] Srivastava, V. K., and Fahim, A., 2007, "An Optimization Method for Solving Mixed Discrete-Continuous Programming Problems," Comput. Math. Appl., **53**(10), pp. 1481–1491.
[33] Geoffrion, A. M., 1972, "Generalized Benders Decomposition," J. Optim. Theory Appl., **10**(4), pp. 237–260.
[34] Westerlund, T., Skrifvars, H., Harjunkoski, I., and Pörn, R., 1998, "An Extended Cutting Plane Method for a Class of Non-Convex MINLP Problems," Comput. Chem. Eng., **22**(3), pp. 357–365.
[35] Pelamatti, J., Brevault, L., Balesdent, M., Talbi, E.-G., and Guerin, Y., 2019, "Efficient Global Optimization of Constrained Mixed Variable Problems," J. Global Optim., **73**(3), pp. 583–613.
[36] Gelbart, M. A., Snoek, J., and Adams, R. P., 2014, "Bayesian Optimization With Unknown Constraints," arXiv preprint arXiv:1403.5607.
[37] Mockus, J., Tiesis, V., and Zilinskas, A., 1978, "The Application of Bayesian Methods for Seaking the Extreme," Towards Global Optim., **2**, pp. 117–129.
[38] Lyu, W., Xue, P., Yang, F., Yan, C., Hong, Z., Zeng, X., and Zhou, D., 2018, "An Efficient Bayesian Optimization Approach for Automated Optimization of Analog Circuits," IEEE Trans. Circuits Systems-I: Regular Paper, **65**(6), pp. 1954–1967.
[39] Lisicki, M., Lubitz, W., and Taylor, G. W., 2016, "Optimal Design and Operation of Archimedes Screw Turbines Using Bayesian Optimization," Appl. Energy, **183**, pp. 1404–1417.
[40] Berk, J., Nguyen, V., Gupta, S., Rana, S., and Venkatesh, S., 2019, "Exploration Enhanced Expected Improvement for Bayesian Optimization," *Machine Learning and Knowledge Discovery in Databases. ECML PKDD 2018*, Lecture Notes in Computer Science, vol. 11052, M. Berlingerio, F. Bonchi, T. Gärtner, N. Hurley, and G. Ifrim, eds., Springer, Cham.
[41] Tao, F., Zhang, M., Liu, Y., and Nee, A. Y. C., 2018, "Digital Twin Driven Prognostics and Health Management for Complex Equipment," CIRP Ann.—Manuf. Technol., **67**(1), pp. 169–172.
[42] Kwong, W. Y., Zhang, P. Y., Romero, D., Moran, J., Morgenroth, M., and Amon, C., 2014, "Multi-objective Wind Farm Layout Optimization Considering Energy Generation and Noise Propagation With NSGA-II," ASME J. Mech. Des., **136**(9), p. 091010.
[43] Savory, E., Parke, G. A. R., Zeinoddini, M., Toy, N., and Disney, P., 2001, "Modelling of Tornado and Microburst-Induced Wind Loading and Failure of a Lattice Transmission Tower," Eng. Struct., **23**(4), pp. 365–375.
[44] Taylor, S. W., Wotton, B. M., Alexander, M. E., and Dalrymple, G. N., 2004, "Variation in Wind and Crown Fire Behaviour in a Northern Jack Pine—Black Spruce Forest," Can. J. Forest Res., **34**(8), pp. 1561–1576.
[45] Belcastro, C. M., and Foster, J. V., 2010, "Aircraft Loss-of-Control Accident Analysis," AIAA Guidance, Navigation, and Control Conference, Toronto, Ontario Canada, Aug. 2–5, pp. 1–41.
[46] Smith, T. G., 2017, "Pmdarima: ARIMA Estimators for Python," https://github.com/alkaline-ml/pmdarima, Accessed January 2020.
[47] Bergmeir, C., and Benítez, J. M., 2012, "On the Use of Cross-Validation for Time Series Predictor Evaluation," Inf. Sci., **191**, pp. 192–213.
[48] Tashman, L. J., 2000, "Out-of-sample Tests of Forecasting Accuracy: An Analysis and Review," Int. J. Forecasting, **16**(4), pp. 437–450.
[49] Wanasinghe, T. R., Wroblewski, L., Petersen, B., Gosine, R. G., James, L. A., De Silva, O., Mann, G. K., and Warrian, P. J., 2020, "Digital Twin for the Oil and Gas Industry: Overview, Research Trends, Opportunities, and Challenges," IEEE Access, **8**, pp. 104175–104197.
[50] Hu, W., Jiang, Z., and Wang, Y., 2018, "Reliability Analysis of Offshore Wind Turbines Using Copula," Proceedings of the ASME 2018 Power and Energy Conference, Lake Buena Vista, FL, June 24–28, pp. 1–8.
[51] Zoph, B., and Le, Q. V., 2016, "Neural Architecture Search With Reinforcement Learning," arXiv preprint arXiv:1611.01578.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/5/051705/6594580/md_143_5_051705.pdf?casa_token=vJFOndGsi5AAAAAA:CzOjbZGfofMEOE4IdDVjkE6meT0oSJL7kQeQE7RMgFjvSYm0muKG5gly5ahvRgSKx7g by Universita Di Pisa user on 28 November